Софийски университет „Св. Климент Охридски“
Факултет по математика и информатика
Катедра „Компютърна информатика“

ДИПЛОМНА РАБОТА

Система за извличане с генериране (RAG) за достъп до фармакотерапевтични ръководства на български език

Цветелина Пашовска
Магистърска програма „Изкуствен интелект“, фак. номер 7MI3400669

Ръководител:
проф. д-р Мария Нишева
катедра „Компютърна информатика“, ФМИ, СУ „Св. Климент Охридски“

София, 2026

Съдържание

1 Увод **2**

1.1 Достъп до медицинска информация на български език и ролята на извличането с генериране . . . 2

1.2 Цели и задачи на дипломната работа . . . 2

2 Литературен преглед **3**

2.1 Архитектура и компоненти на системата за извличане с генериране . . . 3

2.2 Методи за извличане и модели за векторно представяне . . . 4

2.3 Приложения в медицинската област . . . 4

2.4 Особености при работа с по-слабо ресурсни езици . . . 5

3 Методология **6**

3.1 Изходни данни . . . 6

3.2 Изграждане на векторно представяне и индексиране . . . 6

3.3 Компонент за извличане . . . 8

3.4 Генериране на отговори . . . 9

4 Експерименти и резултати **10**

4.1 Демонстрационно приложение . . . 10

4.2 Първа итерация на подобряване на отговорите . . . 13

4.3 Втора итерация на подобряване на системата . . . 17

4.4 Трета итерация: проверка за неподкрепени подробности . . . 24

4.5 Четвърта итерация: маркиране на използваните части от източниците . . . 25

4.6 Качествен анализ и оценка . . . 27

5 Техническа реализация **27**

5.1 Използвани инструменти и технологии . . . 27

5.2 Структура на проекта и файлове . . . 28

5.3 Проверка и възпроизводимост . . . 30

6 Изводи и препоръки за бъдещо развитие **31**

7 Заключение **31**

Приложения **33**

Приложение 1. Инсталиране и стартиране . . . 33

1 Увод

1.1 Достъп до медицинска информация на български език и ролята на извличането с генериране

Фармакотерапевтичните ръководства, публикувани от Националния съвет по цени и реимбурсиране на лекарствените продукти, съдържат клинично и нормативно значима информация за десетки медицински области, но са структурирани като обемни PDF документи с разнородно вътрешно оформление. За здравен специалист, който търси конкретна препоръка, като дозировка, противопоказание или алгоритъм на лечение, това на практика означава ръчно прелистване на десетки страници, дори когато отговорът вече е формулиран в официално одобрен документ. Проблемът тук не е липса на информация, а нейната труднодостъпност в момента на клиничното решение.

Система от типа извличане с генериране (Retrieval-Augmented Generation, RAG) предлага пряко решение на този проблем. Вместо да разчита единствено на параметричната памет на езиков модел, която може да бъде остаряла или неточна за специфична за България нормативна база, системата извлича релевантни откъси от актуалните ръководства и генерира отговор въз основа на тях, придружен от изричен източник: документ, страница и раздел. Именно възможността отговорът да бъде проследен до конкретен официален документ прави подхода особено подходящ за медицински контекст, в който грешен или непроверим отговор носи реален риск, а не само неудобство.

Настоящата дипломна работа надгражда резултатите от преддипломния проект, в който бе изграден и структуриран корпус от 1380 откъса от 17 национални фармакотерапевтични ръководства, готов за индексиране, като всеки откъс носи данни за произход (provenance): документ, страници и раздел. След преглед на научната литература (arXiv, ACL Anthology, RANLP) и общодостъпни източници на български език не беше открита публикувана система от типа извличане с генериране, насочена конкретно към медицински или фармакотерапевтични ръководства на български език. Намерените български публикации по темата са предимно общи, популярни разяснения на подхода, а не описания на реализирана система върху национален медицински корпус. Тази празнина е допълнителна мотивация за настоящата работа: тя не само прилага установен подход към нов, специализиран корпус, но го прави за език и предметна област, за които все още липсва пряк аналог.

1.2 Цели и задачи на дипломната работа

Основната цел на тази дипломна работа е да се разработи работещ прототип на система за извличане с генериране за достъп до национални фармакотерапевтични ръководства на български език, надграждайки резултатите от преддипломния проект с компоненти за извличане на релевантна информация и генериране на отговори.

За постигането на тази цел са поставени следните задачи:

1. аналитичен обзор на литературата по системи за извличане с генериране, методи за извличане (retrieval), приложения в медицинската област и особености при работа с текст на български език;
2. изграждане на векторно представяне (embeddings) на структурираните в преддипломния проект текстови откъси с многоезичен модел, подходящ за българския език,

и индексиране във векторна база от данни;
3. реализация на компонент за извличане на релевантна информация (retrieval) по зададена заявка;
4. проектиране и имплементиране на слой за генериране на отговори на база извлечен контекст (prompt engineering);
5. разработване на демонстрационно уеб приложение за задаване на въпроси и преглед на генерираните отговори;
6. качествен анализ на примерни резултати от системата, включващи въпроси от различни медицински области, и дискусия на наблюдаваните ограничения;
7. формулиране на изводи и препоръки за бъдещо развитие.

2 Литературен преглед

2.1 Архитектура и компоненти на системата за извличане с генериране

Подходът извличане с генериране (Retrieval-Augmented Generation, RAG) е предложен от Lewis и др. [1] като начин да се съчетаят параметричната памет на голям езиков модел с непараметрична памет под формата на външен, обновяем корпус от документи: вместо да разчита единствено на теглата си, моделът получава по заявка извлечени откъси и генерира отговор въз основа на тях. Тази идея решава едновременно два практически проблема: ограничената и статична памет на езиковите модели и невъзможността отговорът да бъде проверен спрямо конкретен източник. Същевременно обаче тя прехвърля тежестта на верността на отговора върху качеството на извличането, а не само върху генеративния модел.

По-новите систематизиращи прегледи очертават еволюция от тази първоначална, линейна схема, при която заявката преминава последователно през извличане, допълване на подсказката и генериране, към т.нар. модулна архитектура, при която всеки компонент – индексиране, извличане, преработка на извлечения контекст, генериране – е независимо заменяем и може да се комбинира с допълнителни стъпки, като преформулиране на заявката или повторно класиране на резултатите [2]. Sharma [3] систематизира съществуващите варианти в няколко ориентации, насочени съответно към подобряване на извличането, на генеративния модул, към хибридни решения и към устойчивост спрямо шумни или противоречиви извлечени откъси, и посочва качеството на извличането като най-честото тясно място в цялостната система. Като по-нататъшно развитие се откроява агентният подход (agentic RAG), при който системата итеративно решава дали и какво да извлече, вместо да следва фиксирана еднократна последователност [4]. За целите на настоящата дипломна работа подобна сложност не е оправдана, тъй като въвежда допълнителна латентност и трудности при проследяване на произхода на отговора, без да е ясно, че ще подобри резултата върху затворен и сравнително добре структуриран корпус от национални ръководства. Затова избраната отправна точка тук е модулна, но не агентна архитектура, съобразена със структурата, зададена от преддипломния проект: индексиране на откъсите, самостоятелен компонент за извличане и отделен слой за генериране, всеки от които може да бъде подобряван независимо.

Оценяването на подобни системи е отделен методологически въпрос: освен показателите за самото извличане (виж 2.2), генерираният отговор трябва да бъде проверен

за вярност спрямо извлечения контекст, тъй като езиковият модел може да „халюцинира“ твърдения, които не се съдържат в подадените откъси. Malin и др. [5] систематизират съществуващите показатели за вярност (faithfulness) и отбелязват, че оценяването с друг езиков модел като съдия към момента корелира най-добре с човешката преценка, докато Tamber и др. [6] предлагат постоянно обновяван еталон именно за тази цел. Тези резултати мотивират методиката за качествен анализ в тази дипломна работа (виж раздел 4.6) да включва не само проверка на извлечения източник, но и отделна преценка доколко генерираният отговор се придържа само към предоставения контекст.

2.2 Методи за извличане и модели за векторно представяне

Класическото разграничение в извличането на информация е между лексикално извличане, което съпоставя точни думи или n-грами (например BM25), и семантично извличане, при което заявката и документите се представят като вектори в общо пространство и се сравняват чрез мярка за близост, най-често косинусова близост, независимо от точното съвпадение на думите. Лексикалното извличане е бързо, обяснимо и не изисква обучение, но пропуска синоними и перифрази. Семантичното извличане улавя смисъла, но зависи изцяло от качеството на модела за векторно представяне (embeddings) и е по-трудно за диагностициране, когато върне грешен резултат. За медицински текст, богат на синоними, съкращения и специализирана терминология, например различни търговски и генерични названия на едно и също лекарство, семантичното извличане изглежда по-подходящо. Именно поради тази специализирана терминология обаче качеството на векторното представяне не бива да се приема за дадено, а трябва да бъде проверено емпирично (виж раздел 3.2).

Що се отнася до самите модели за векторно представяне, Luo и др. [7] показват, че използването на голям езиков модел като енкодер за плътно извличане (dense retrieval) подобрява точността спрямо по-малки специализирани енкодери, но с чувствително по-висока изчислителна цена. Този компромис трябва да се претегли спрямо ограничените изчислителни ресурси на дипломна работа и необходимостта системата да отговаря в разумно време при демонстрация. Извличането по правило се комбинира с векторна база от данни, оптимизирана за приблизително търсене на най-близки съседи. Неотдавнашно емпирично изследване на седем широко използвани системи – FAISS, Qdrant, Milvus, Weaviate, Chroma, pgvector и LanceDB – показва съществени разлики в компромиса между точност, латентност и консумация на памет в зависимост от индексиращата структура [8]. Това налага изборът на векторна база в тази дипломна работа (раздел 3.2) да бъде направен според мащаба на корпуса от преддипломния проект, тоест 1380 откъса, а не според общи препоръки за много по-големи колекции.

2.3 Приложения в медицинската област

Систематизиращият преглед на He и др. [9] обхваща технологиите, наборите от данни и клиничните приложения на извличането с генериране в биомедицината и подчертава повтарящ се проблем: разминаване между способността на системите да извличат релевантна информация и способността им да я обосноват по проверим начин пред клиницист. Скринингов преглед на Miao и др. [10] върху 67 проучвания в областта на медицината и сестринските грижи установява, че по-голямата част от съществуващите системи са насочени към отговаряне на медицински въпроси и клинична подкрепа при вземане на реше-

ния. Приложения, изградени върху затворен корпус от официално одобрени ръководства, какъвто е разглежданият в тази работа, остават сравнително слабо застъпени: повечето изследвани системи разчитат на широки, отворени биомедицински корпуси, например PubMed, а не на регулаторно одобрен, ограничен по обхват източник.

Именно тази разлика е показателна за настоящата дипломна работа. Докато извличането от отворен уеб корпус трябва допълнително да преценява достоверността и авторитетността на всеки източник, затвореният корпус от национални фармакотерапевтични ръководства носи авторитетност „по подразбиране“, а основният риск се измества към точността на самото извличане и към честността на генеративния модул да не добавя информация извън предоставения контекст. Li и Zhu [11] формулират именно такива практически препоръки от индустриален опит за медицински системи за извличане с генериране: стриктно ограничаване на генерирания отговор до предоставения контекст, връщане на изричен източник (документ, страница, раздел) към всяко твърдение и изрично сигнализиране, когато не е намерен релевантен откъс. Тези препоръки директно съответстват на изискването за данни за произход, наследено от преддипломния проект, и са отправна точка при проектирането на слоя за генериране на отговори (раздел 3.4), където е обсъдено и доколко могат да бъдат изпълнени от малък генеративен модел.

2.4 Особености при работа с по-слабо ресурсни езици

Повечето цитирани дотук резултати са получени и оценени основно върху английски текст, което поставя въпроса доколко изводите им важат за българския език. Съществуват по същество два алтернативни подхода за извличане на информация, когато езикът на корпуса е по-слабо ресурсен. Първият е кросезиково извличане: заявката или корпусът се превежда към високоресурсен език като английски, след което се използва силен английски модел за извличане и генериране. Ranaldi и др. [12] показват, че подобен превод преди извличането подобрява резултатите върху задачи, изискващи много фактическо знание, включително за по-слабо ресурсни езици. Този подход обаче въвежда допълнителна стъпка на превод, която може да изкриви медицинска терминология, дозировки и съкращения. Рисковете от подобно изкривяване вече бяха отчетени при OCR обработката в преддипломния проект и биха се повторили при машинен превод. Вторият подход е директна работа на изходния език чрез многоезичен или езиково специфичен модел за векторно представяне, без междинен превод. Той запазва оригиналната терминология, но изисква съответният модел действително да представя добре българския език, а не само големите езици в обучаващата му извадка.

По втория подход има насърчителни, но неоднозначни резултати. От една страна, съществуват специализирани български модели от типа BERT и RoBERTa, обучени основно за класификация, разпознаване на именувани обекти и синтактичен анализ [13, 14]. Това показва, че за българския език вече има зряла инфраструктура за представяне на текст, но тези модели не са били оценявани конкретно за задачата извличане чрез сходство на вектори, което представлява отворен въпрос. От друга страна, изследвания върху други по-слабо ресурсни езици показват, че общите многоезични модели за векторно представяне често подценяват съответния език и се нуждаят от допълнително адаптиране. Omotoso и др. [15] дообучават многоезичния модел BGE-M3 за нигерийски по-слабо ресурсни езици с осезаемо подобрение. Nzeyimana и Rubungo [16] изграждат специализиран, лексикално обоснован модел за извличане за езика киняруанда вместо да разчитат на генеричен многоезичен модел. Navasardyan и др. [17] показват, че дори малък обем шумни, синтетично

генерирани данни е достатъчен за забележимо подобряване на векторното представяне на по-слабо ресурсен език. Общото между тези резултати е изводът, че годността на дадено многоезично векторно представяне за конкретен по-слабо ресурсен език не бива да се приема по подразбиране, а трябва да се провери емпирично и, при нужда, да се коригира с целево, дори ограничено по обем, дообучение.

За разлика от векторното представяне, при генеративния компонент положението за българския език е по-благоприятно. Alexandrov и др. [18] от Института по компютърни науки, изкуствен интелект и технологии (INSAIT) представят BgGPT – семейство отворени модели, получени чрез продължено предобучение на Gemma 2 върху около 100 милиарда токена, от които 85 милиарда на български, съчетано с техника за сливане на модели (branch-and-merge), ограничаваща катастрофалното забравяне на английски и общи умения при адаптирането към нов език. Моделите се разпространяват в няколко размера – 2.6, 9 и 27 милиарда параметри – с отворени тегла и готови квантувани версии за локално изпълнение, а по данни на самите автори по-малкият вариант надминава по представяне на български сравними по размер отворени модели. Наличието на специализиран за българския език генеративен модел, който може да се зарежда изцяло локално, прави обосновано генеративният компонент на настоящата система (раздел 3.4) да се базира на BgGPT, вместо на по-голям, но неспециализиран за българския език модел или на модел, достъпен само през платено API.

От направения преглед следват две методологически решения за настоящата дипломна работа. Първо, вместо предварително да се избере един модел за векторно представяне въз основа само на общите му многоезични показатели, ще бъдат съпоставени емпирично няколко кандидат-модела върху корпуса от преддипломния проект (раздел 3.2), а директният подход на изходния език ще бъде предпочетен пред превод към английски, за да се запази медицинската терминология непроменена. Второ, за генериране на отговори (раздел 3.4) ще бъде използван BgGPT, зареден локално чрез Ollama, което съответства едновременно на изискването системата да работи без връзка към платено API и на установената по-добра пригодност на този модел за българския език.

3 Методология

3.1 Изходни данни

Корпусът е резултат от преддипломния проект и се състои от 1380 откъса (chunks), получени чрез разбиване на текста на фрагменти (chunking) от 17 национални фармакотерапевтични ръководства. Всеки откъс запазва данни за произход: документ, диапазон страници, секция и идентификатор (`chunk_id`).

3.2 Изграждане на векторно представяне и индексиране

В съответствие с извода от литературния преглед (раздел 2.2), моделът за векторно представяне не е избран предварително, а се съпоставят три отворени, многоезични кандидата, поддържащи българския език, всеки изпълняван локално чрез библиотеката `sentence-transformers` без връзка към платено API: `BAAI/bge-m3` [19], `intfloat/multilingual-e5-base` [20] и `sentence-transformers/paraphrase-multilingual-mpnet-base-v2` [21]. При моделите от семейството e5 текстовете се пред-

ставят с префиксите `query:` и `passage:`, изисквани от авторите им за коректна работа, докато другите два модела не се нуждаят от такъв префикс – разлика, която се третира изрично в имплементацията (`src/medguiderag/indexing/embed.py`), за да не се получи подвеждащо ниска прилика само поради пропусната конвенция.

За векторна база е избран FAISS, а не сървър-базирано решение като Qdrant или Milvus, разгледани в раздел 2.2: при корпус от 1380 откъса точното (а не приближено) търсене чрез `IndexFlatIP` върху нормализирани вектори – еквивалентно на косинусова близост – отнема пренебрежимо малко време, така че компромисите между точност и латентност при приближено търсене на най-близки съседи, обсъдени по-горе, тук не се проявяват. За всеки кандидат-модел се изгражда отделен индекс с придружаващи метаданни (документ, страници, секция, текст на откъса), което позволява последващо сравнение един срещу друг върху едни и същи заявки (раздел 4.6).

Таблица 1 обобщава резултата от индексирането на трите кандидата върху локалната машина (на централния процесор, ограничен до две нишки на процес, за да се избегне изчерпване на паметта). Времето за изчисляване на векторните представяния е еднократен, офлайн разход при построяването на индекса и не се отразява на латентността при отговор на заявка, но е практически показателен за скоростта на итериране по време на разработка: `BAAI/bge-m3` отнема близо два часа за целия корпус, близо 20 пъти повече от по-леките кандидати. Разликата произтича от по-големия му размер (568 милиона параметри спрямо около 278 милиона за другите два) и по-високата размерност на изхода.

Таблица 1: Резултати от индексирането на трите кандидат-модела върху корпуса от 1380 откъса

Модел	Измерения	Размер на индекса	Време за индексиране
paraphrase-multilingual-mpnet-base-v2	768	4.0 MB	5.9 мин
multilingual-e5-base	768	4.0 MB	15.9 мин
BAAI/bge-m3	1024	5.4 MB	115.9 мин

Разликата във времето за индексиране сама по себе си не определя кой модел е по-подходящ; това ще бъде установено едва чрез качеството на извличането върху ръчно анотирани въпроси (раздел 4.6). Тя е основание обаче изборът между кандидатите да отчита и практическата цена на преизчисляване на индекса, ако корпусът или стратегията за фрагментиране се променят по-нататък в разработката. След преминаването към изчисления на графичния процесор във втората итерация (раздел 4.3) индексирането с `BAAI/bge-m3` отнема 84 секунди, така че тази цена престава да бъде практическо ограничение.

Трите кандидата се различават и по характеристика, която не личи от таблица 1, но се оказва решаваща: максималната дължина на входа, която моделът обработва при изчисляване на векторното представяне. Тя е 8192 токена за `BAAI/bge-m3`, 512 за `intfloat/multilingual-e5-base` и едва 128 за `paraphrase-multilingual-mpnet-base-v2`, а подългите текстове се отрязват без предупреждение. Откъсите от преддипломния проект са с дължина около 420 думи, т.е. средно около 860 токена, като размерът им е избран преди да е известен моделът за векторно представяне. Последиците от това несъответствие са измерени в раздел 4.2 и показват, че сравнението на трите кандидата при сегашния размер на откъсите не е равностойно.

3.3 Компонент за извличане

Компонентът за извличане приема текстова заявка и връща най-много k откъса заедно с оценка на релевантността и пълните данни за произход. В сегашния си вид той е хибриден и се състои от три последователни стъпки (`src/medguiderag/retrieval/hybrid.py`):

1. *Кандидати.* Заявката се търси по два независими начина: семантично, чрез векторното представяне и индекса от раздел 3.2, и лексикално, чрез функцията за класиране BM25 [22] върху същите откъси. Преди лексикалното търсене всяка дума се съкращава до първите си шест знака, за да съвпадат различните граматични форми на една и съща дума. Всеки от двата метода връща по 50 откъса.
2. *Сливане.* Двата списъка се обединяват чрез сливане по реципрочни рангове (Reciprocal Rank Fusion) [23]: всеки откъс получава сумата от $1/(60 + r)$ за всеки от списъците, в които се среща на позиция r, и за следващата стъпка се запазват първите 20.
3. *Повторно класиране (reranking).* Двадесетте кандидата се подреждат наново от модела `BAAI/bge-reranker-v2-m3`, изграден върху `BAAI/bge-m3`. За разлика от модела за векторно представяне, той обработва заявката и откъса заедно, като кръстосан енкодер (cross-encoder) [24], и връща оценка на релевантността между 0 и 1. Запазват се най-много k откъса с оценка поне 0.1. Ако нито един откъс не достига този праг, системата не генерира отговор, а съобщава, че в ръководствата не е открита информация по въпроса.

Всяка от стъпките отговаря на конкретна слабост на по-простите алтернативи, измерена във втората итерация (раздел 4.3). Лексикалното търсене е добавено, защото двата метода грешат при различни въпроси и обединението им съдържа верния откъс по-често от всеки поотделно. За съкращаването на думите е предпочетено грубо отрязване пред морфологичен анализатор за български, защото не изисква допълнителен езиков ресурс, а вече подобрява съществено лексикалното търсене. Сливането използва само позициите в списъците, а не самите оценки, защото оценките на BM25 и косинусовата близост са в несравними скали. Претеглена сума от двете оценки би изисквала настройване на тегло върху данни, каквито в достатъчен обем липсват, докато константата 60 е стойността, препоръчана от Cormack и др., и не е настройвана. Кръстосаният енкодер е по-точен от сравнението на независимо изчислени вектори, защото вижда заявката и откъса едновременно, но е и много по-бавен: оценяването на всичките 1380 откъса за всяка заявка би било непрактично, затова той се прилага само върху малък брой кандидати. Оценката му е в една и съща скала за всички заявки и затова служи и като праг за отказ. Така изискването за изрично съобщение при липса на релевантна информация (раздел 3.4) се изпълнява на ниво извличане, а не се поверява на генеративния модел.

Първоначалният вариант – само семантично търсене и k най-близки откъса (`src/medguiderag/retrieval/retriever.py`) – е запазен като алтернативен режим, който се избира от интерфейса. Той служи за отправна точка при сравненията и е единственият практичен режим, когато няма графичен процесор, тъй като на централния процесор повторното класиране отнема около 70 секунди на въпрос. И в двата режима всеки от моделите за векторно представяне от раздел 3.2 се използва през един и същ интерфейс, което позволява сравняването им без промяна на останалата част от системата.

3.4 Генериране на отговори

За генеративния компонент е избран BgGPT (раздел 2.2) във варианта с 2.6 милиарда параметри и 4-битово квантуване, зареден локално чрез Ollama, без връзка към платено API. По-големите варианти (9 и 27 милиарда параметри) изискват повече памет от наличните 6 GB на видеокартата, а 5-битовото квантуване на по-малкия модел не успя да бъде заредено изцяло на нея, въпреки че формално свободната видеопамет е достатъчна. По-новият вариант на BgGPT с 4 милиарда параметри, изграден върху Gemma 3, също с 4-битово квантуване, е сравнен с избрания в раздел 4.3 и след въвеждането на проверката за неподкрепени подробности е приет по подразбиране (раздел 4.4). По-малкият модел остава избираем от интерфейса.

Подсказката към модела е съставена по следния начин, като всяко от решенията е резултат от експериментите в раздел 4.2:

- Инструкцията и откъсите се подават в едно потребителско съобщение, а инструкцията стои след откъсите, непосредствено преди въпроса. Шаблонът за диалог (chat template) на моделите Gemma 2 и Gemma 3, върху които са изградени двата варианта на BgGPT, няма отделна системна роля, и системното съобщение се подава като допълнителен потребителски ход в самото начало, най-далеч от въпроса.

- Откъсите се подреждат в обратен ред на релевантността, така че най-близкият до заявката откъс е последен, непосредствено до въпроса.

- Текстът на откъсите се нормализира: преносите на редове от PDF документите, които при двустранно подравнените редове отделят дори единични думи, се заменят с интервали.

- Инструкцията изисква отговор само с информация от откъсите, без добавени факти, но не изисква цитиране на източници. Източниците се показват от системата детерминистично, от данните за произход на извлечените откъси, тъй като моделът измисля цитати, когато бъде помолен да ги посочи.

- Контекстният прозорец (context window) е зададен изрично на 8192 токена, колкото е естественият за Gemma 2, вместо стойността по подразбиране на Ollama от 4096 токена, при която подсказката се отрязва без предупреждение. Поради същото ограничение броят на подаваните откъси е най-много пет.

- Температурата на генериране, т.е. параметърът, който управлява степента на случайност при избора на следващ токен, е нула. Така един и същ въпрос върху едни и същи откъси получава практически един и същ отговор, което е предпоставка за сравнимост на експериментите. Остатъчните разлики между прогоните са обсъдени в раздел 4.3.

- Наказанието за повторение (repeat penalty) е 1.2, а дължината на отговора е ограничена до 1024 токена. Без тях моделът може да зацикли, когато получи само един откъс (раздел 4.3).

- След генерирането отговорът се проверява автоматично за имена на лекарства, АТС кодове и числа, които не се срещат в откъсите, и те се отбелязват в интерфейса (раздел 4.4).

- Изреченията от отговора се съпоставят с изреченията от откъсите, а използваните части се маркират в показаните източници (раздел 4.5).

Спрямо препоръките на Li и Zhu [11] това е съзнателно отклонение. Вместо моделът да посочва източник към всяко твърдение, проследимостта се постига на ниво система: потребителят вижда точните откъси, върху които е изграден отговорът, заедно с документа и страниците им. Изискването за изрично съобщение при липса на релевантна информация също не е поверено на модела, защото при опит да бъде изпълнено чрез инструкция моделът отказва да отговори и когато откъсите съдържат отговора. Решението дали изобщо има релевантен откъс е по-естествено да се взема на ниво извличане и във втората итерация е реализирано чрез праг върху оценката на повторното класиране (раздел 3.3).

4 Експерименти и резултати

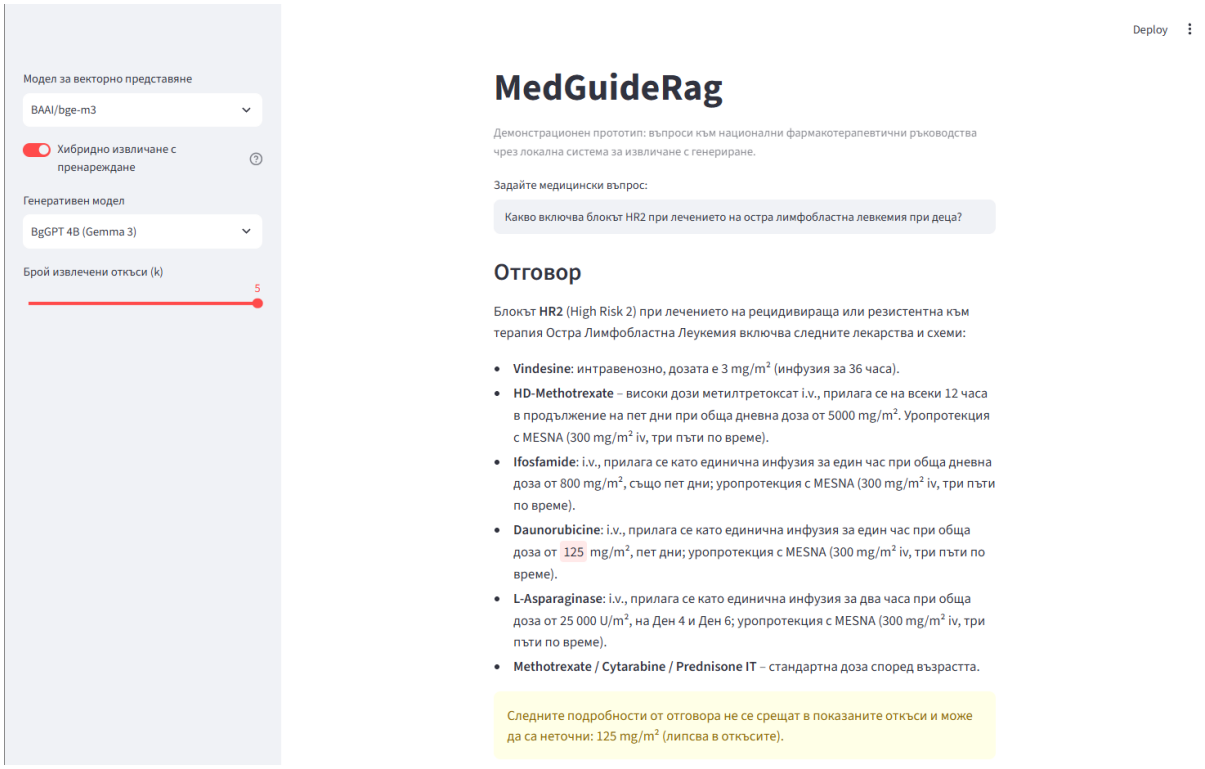
4.1 Демонстрационно приложение

Демонстрационното приложение дава достъп до системата чрез уеб интерфейс, в който потребителят задава въпрос и вижда отговора заедно с откъсите, на които той стъпва. Описано е окончателното му състояние, а промените, довели до него, са обосновани в итерациите от следващите раздели. Приложението е реализирано с библиотеката Streamlit (`src/medguiderag/app/ui.py`). Разгледани са три алтернативи. Gradio също позволява бързо изграждане на интерфейс към модели, но е насочен преди всичко към демонстрации с един вход и един изход, докато тук резултатът се състои от отговор, предупреждения и няколко източника с маркиран текст. Отделен сървър (например Flask или FastAPI) със собствен клиент на JavaScript дава пълен контрол върху интерфейса, но изисква втори език и двойно повече код, което не е оправдано за прототип с един потребител. Chainlit е предназначен за разговор с история, какъвто приложението съзнателно не поддържа. Streamlit позволява целият конвейер да остане на Python. Моделите за векторно представяне и за повторно класиране и данните за проверката на отговора се зареждат веднъж, при първия въпрос, и се преизползват при следващите.

Интерфейс. Въпросът се въвежда в едно текстово поле и всеки въпрос се обработва самостоятелно, без история на разговора. Историята би заела част от контекстния прозорец от 8192 токена, който петте откъса почти изчерпват (раздел 3.4), и би смесила откъси от различни въпроси. Освен това проверката и маркирането на източниците се отнасят до откъсите на текущия въпрос. Отговорът се показва под въпроса (фигура 1). Имената на лекарства, АТС кодовете и числата, които не се срещат в откъсите, са оцветени и изброени в предупреждение под отговора (раздел 4.4). Отговорът е придружен и от постоянна бележка, че е генериран от езиков модел и може да съдържа неточности.

Под отговора са източниците (фигура 2). Всеки откъс е показан като разгъващ се панел, озаглавен с документа, страниците и оценката на релевантност. В заглавието е посочено и колко изречения от откъса са маркирани или че откъсът не е използван в отговора. В панела са разделът на ръководството и пълният текст на откъса, в който са оцветени изреченията, съпоставени с отговора (раздел 4.5). Изреченията от отговора, за които не е открито съответствие, са изброени в отделен панел. Ако нито един откъс не достигне прага

на релевантност, системата не генерира отговор, а показва съобщение, че в ръководствата не са открити откъси по въпроса (фигура 3 и раздел 4.3).

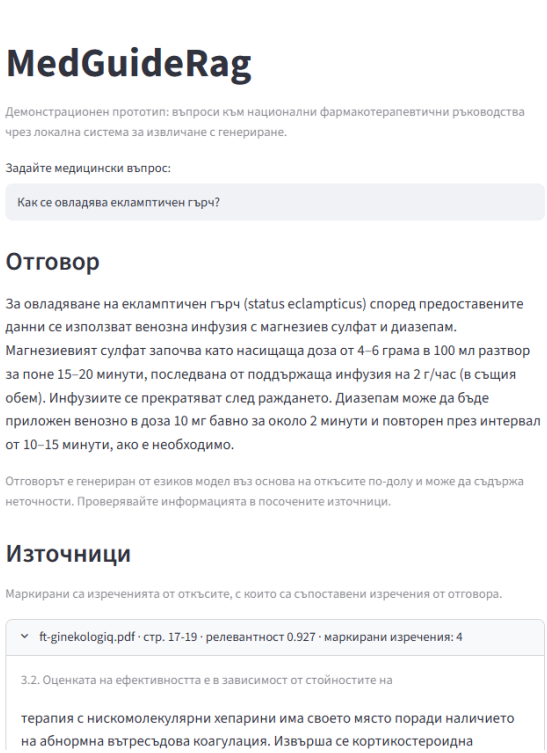


Фигура 1: Отговор с отбелязана неподкрепена подробност и предупреждение под него. Вляво са настройките на приложението.

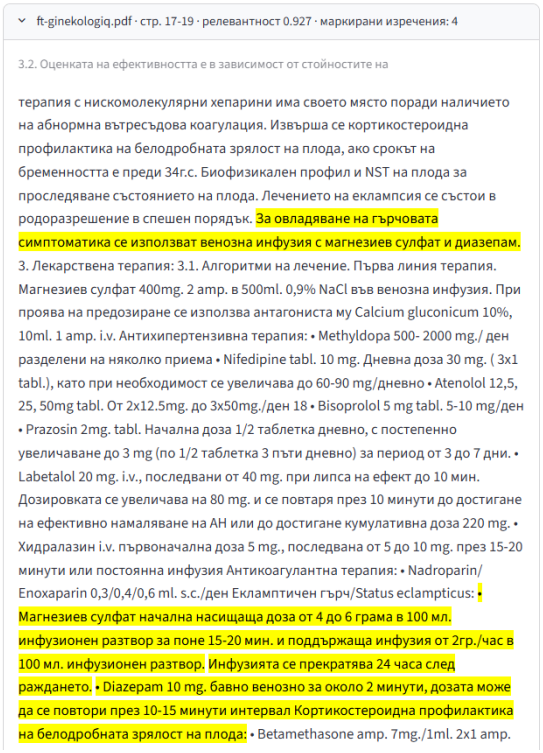
Настройки. Страничната лента съдържа четири настройки, които позволяват част от сравненията в следващите раздели да се повторят направо в интерфейса:

- *Модел за векторно представяне* – един от трите кандидата от раздел 3.2, по подразбиране `BAAI/bge-m3`.
- *Хибридно извличане с повторно класиране* – включено по подразбиране, когато е наличен графичен процесор. Изключено, системата използва само семантичното търсене и показва косинусовата близост вместо оценката на релевантност. Възможността е нужна, защото без графичен процесор повторното класиране отнема около 70 секунди на въпрос.
- *Генеративен модел* – BgGPT с 4 милиарда параметри по подразбиране или вариантът с 2.6 милиарда (раздел 4.4).
- *Брой откъси* – от един до пет. Горната граница следва от контекстния прозорец, а при хибридното извличане откъсите могат да са по-малко, защото тези под прага се отхвърлят.

Обработка на въпрос. След въвеждането на въпроса приложението последователно извлича откъсите (семантично и лексикално търсене, сливане, повторно класиране, праг; раздел 3.3), генерира отговора чрез Ollama, проверява го за неподкрепени подробности и съпоставя изреченията му с откъсите. Първият въпрос отнема около 35 секунди, защото



(а) отговор



(б) разгънат първи източник

Фигура 2: Отговор и разгънатият му първи източник: изреченията от откъса, съпоставени с изречения от отговора, са оцветени.

тогава моделите се зареждат на графичния процесор. Следващите отговори отнемат около 6–8 секунди с модела с 4 милиарда параметри и около 5 секунди с по-малкия, а отказът около 2 секунди. При заредени модели за векторно представяне, за повторно класиране и генеративен модел видеопаметта е заета до около 5.7 от 6 GB. Затова в нея се побира само един генеративен модел, а смяната му в страничната лента отнема около 10 секунди за повторно зареждане.

Локално изпълнение. Всички компоненти работят на машината, без връзка към външни услуги. Ollama приема заявки само на локалния адрес, а моделите за векторно представяне и за повторно класиране се зареждат от локалното хранилище на Hugging Face в режим без мрежа. Без този режим библиотеката проверява при всяко зареждане дали в хранилището има по-нова версия на модела. Проверка на мрежовите връзки по време на зареждането потвърждава, че в режим без мрежа приложението не осъществява нито една връзка извън машината, срещу една връзка без него.

Ограничения. Приложението е предназначено за един потребител. Моделите се споделят между отворените сесии, но заявките към тях се обработват последователно, а видеопаметта не оставя място за паралелна работа. Отговорът се показва след като е генериран изцяло, а не постепенно, докато се генерира. Приложението не поддържа вход с потребителско име, история на въпросите или събиране на обратна връзка. Сървърът на Streamlit е достъпен по подразбиране и от локалната мрежа, затова при демонстрация на споделена мрежа достъпът трябва да се ограничи до самата машина.



Фигура 3: Отказ при въпрос извън обхвата на ръководствата: нито един откъс не достига прага и генеративният модел не се използва.

4.2 Първа итерация на подобряване на отговорите

Първите опити с демонстрационното приложение показаха, че отговорите често не възпроизвеждат информацията от показаните източници, а съдържат общи или неподкрепени от тях твърдения. Характерен пример е въпросът „Как се лекува хиперемезис гравидарум?“ с модела за векторно представяне `BAAI/bge-m3`. Извлеченият откъс от Фармакотерапевтичното ръководство по акушерство и гинекология изрежда конкретни групи лекарства: антиеметици (ondansetron, metoclopramide, фенотиазини), антисекреторни средства (esomeprazole, famotidine, ranitidine), инфузии с глюкозни разтвори и електролити, седативни средства. Генерираният отговор споменава инфузионната терапия и антисекреторните средства, но пропуска антиеметиците и добавя групи лекарства, каквито в откъса няма: антихипертензивни средства (метопролол), антикоагуланти (варфарин) и антиконвулсанти (валпроат). Разделът описва експериментите, с които бяха установени причините, и промените, които подобриха отговорите. Промените не са окончателни и разделът завършва с проблемите, които остават за следващата итерация.

Протокол. За да се отделят отделните причини, експериментите следват един и същ протокол. Използват се три въпроса от различни области, за които корпусът съдържа отговор: хиперемезис гравидарум (акушерство и гинекология), кога се прилага инсулин при захарен диабет тип 2 (ендокринология) и лечение на атопичен дерматит (дерматология). За всеки въпрос откъсите се извличат веднъж с `BAAI/bge-m3` ($k = 5$) и остават фиксирани, а между вариантите се променя по един фактор на подсказката. Генерирането е с температура нула, за да се изключи случайността. Всеки отговор се оценява по два начина. Автоматично се брои колко от предварително изброените факти в релевантния откъс се срещат в отговора (съответно 14, 7 и 8 факта, най-често имена на лекарства и мерки). Ръчно отговорът се прочита изцяло и се отбелязват твърденията, които не се съдържат в откъсите. Автоматичната мярка е груба, защото отчита само наличието на ключова дума, и при разминаване предимство има ръчният прочит.

Отрязване на подсказката. Първата находка не е свързана с формулировката на подсказката, а с размера ѝ. Пет откъса заедно с инструкцията заемат между 5500 и 6300 токена, а Ollama по подразбиране използва контекстен прозорец от 4096 токена. При препълва-

не библиотеката llama.cpp, чрез която Ollama изпълнява модела, запазва само последната половина от прозореца. Така подсказката е била съкратена до около 2000 токена, без предупреждение, като отпадат първите откъси, които при подреждане по релевантност са и най-важните. Отрязването е установено по броя обработени токени, който Ollama отчита за всяка заявка. Контекстният прозорец е увеличен до 8192 токена, а броят на откъсите е ограничен до пет.

Системна роля, цитиране и отказ. Първоначалната подсказка съдържаше правилата в системно съобщение: да се използва само предоставеният контекст, да се цитира източник във формат „[документ, стр.]“ и изрично да се съобщава, когато липсва информация. Тъй като шаблонът за диалог на Gemma 2 няма системна роля, правилата попадат в отделен потребителски ход в самото начало, а при описаното по-горе отрязване на подсказката те са отпадали изцяло. Затова правилата бяха преместени непосредствено преди въпроса, а контекстният прозорец беше увеличен. Проверката на този вариант с откъси, извлечени с `paraphrase-multilingual-mpnet-base-v2`, разкри два по-сериозни проблема. С дословно зададена фраза за отказ моделът отговаря с нея на всички въпроси, включително на въпроса за инсулина, чийто отговор се съдържа във втория извлечен откъс. Без тази фраза моделът отговаря, но измисля източници: за контролния въпрос „Колко е висок връх Мусала?“, на който корпусът няма отговор, посочва хематологичното ръководство, а в друг вариант несъществуващ файл „ФТР_География.pdf“. В медицински контекст измисленият източник е по-опасен от липсващия, защото придава привидна достоверност на отговора. Затова цитирането и отказът са изнесени извън генеративния модел (раздел 3.4).

Варианти на подсказката. Таблица 2 обобщава вариантите, сравнени при фиксирани откъси. Изходният вариант V0 съответства на състоянието на системата в момента на наблюдавания проблем: инструкцията е след откъсите, а контекстният прозорец е 8192 токена.

Таблица 2: Възпроизведени факти от релевантния откъс при различни варианти на подсказката (откъси, извлечени с `BAAI/bge-m3`, температура нула)

Вариант	Описание	Хиперемезис	Инсулин	Дерматит
V0	изходен: необработен текст, 5 откъса по реда на релевантност	2/14	1/7	3/8
V1	V0 с нормализиран текст	1/14	1/7	3/8
V2	V1, най-релевантният откъс последен	7/14	2/7	4/8
V3	V2 с 3 откъса	7/14	3/7	4/8
V4	V2 с конкретизираща инструкция	9/14	5/7	4/8
V5	V4 с 3 откъса	8/14	6/7	4/8
V6	само най-релевантният откъс, конкретизираща инструкция	7/14	4/7	4/8

Ред на откъсите. Най-голямото подобрение идва от реда на откъсите (V1 спрямо V2): при въпроса за хиперемезис възпроизведените факти нарастват от 1 на 7 от 14. При подреждане по релевантност най-подходящият откъс е първи, на около 5000 токена от въпроса, и моделът на практика го пренебрегва, като съставя отговора от последните, несвързани откъси и от собствената си параметрична памет. Във варианта V1 например отговорът

представя като лечение на хиперемезис вазоактивните лекарства terlipressin и octreotide и инфузиите със somatostatin. Те са от последния откъс, от ръководството по гастроентерология, който описва лечението на кървене. Резултатът е в съгласие с наблюденията на Liu и др. [25], според които езиковите модели използват най-добре информацията в началото и в края на дълъг контекст и най-слабо тази в средата. При BgGPT с 2.6 милиарда параметри ефектът е дори по-силен: в наблюдаваните случаи позицията в началото не е достатъчна, ако между нея и въпроса има хиляди токени несвързан текст, и решаваща е близостта до въпроса.

Нормализиране на текста. Замяната на преносите на редове с интервали (V0 спрямо V1) променя броя възпроизведени факти най-много с един, но съкращава подсказката с около 6% и премахва накъсването на изреченията, затова е запазена.

Конкретизираща инструкция. Инструкция, която изрично иска да се изброят мерките, лекарствата и дозировките, посочени в откъсите (V4–V6), повишава автоматичната оценка при хиперемезис и при инсулина, но ръчният прочит показва обратното. Моделът попълва исканата структура с измислено съдържание. При хиперемезис добавя диетичен режим („избягване на мазни и пикантни храни“, „банани, ориз и ябълково пюре“), какъвто в откъса няма, приписва на лекарствата погрешни ATC кодове, а във варианта V5 посочва и дози („Esomeprazole: 20 mg перорално дневно“, „Ondansetron: 4 mg“), каквито откъсът не съдържа. При инсулина посочва взаимно противоречиви прагове на гликирания хемоглобин. Затова е запазена по-простата инструкция. Случаят илюстрира и ограничението на автоматичната мярка: тя отчита наличието на ключова дума, но не и верността на твърдението около нея.

Брой откъси. С три откъса (V3) отговорите остават верни спрямо източниците, броят възпроизведени факти не намалява, а подсказката е с около 40% по-кратка. Въпреки това е запазен вариантът с пет откъса, защото при три откъса верният откъс отпада винаги, когато извличането го класира на четвърто или пето място. В набора за разработка от раздел 4.3 това се случва при два от 24 въпроса.

Температура. При температура 0.2 три последователни изпълнения на въпроса за инсулина върху едни и същи пет откъса (извлечени с `paraphrase-multilingual-mpnet-base-v2`) дадоха три различни отговора, единият с невярно твърдение: хипогликемията е представена като показание за инсулиново лечение, докато тя е риск от него. Температура нула прави отговорите повторими и е условие за сравнимост на експериментите.

Резултат. След промените (вариант V2) отговорът за хиперемезис описва инфузиите с глюкозни и левулозни разтвори с добавен инсулин, корекцията на електролитите, поливитамините, групите антиеметици (5-HT3-блокери, D2-рецепторни блокери, H1-блокери), антисекреторните средства (esomeprazole, famotidine, ranitidine) и седативните средства, както и критериите за оценка на ефективността. Всички тези сведения се съдържат в откъса. Пропуснати остават хоспитализацията, покоят и диетичните препоръки от първоначалните грижи, както и конкретните антиеметици.

Зависимост от извличането. Промените в подсказката подобряват отговора само когато релевантният откъс е сред извлечените. Таблица 3 показва, че при въпроса за хиперемезис това е изпълнено единствено за `BAAI/bge-m3`. Другите два модела не извличат откъса с отговора сред първите пет, а връщат откъси от ръководствата по нуклеарна медицина, нефрология, гастроентерология, ендокринология, хематология, неврология и неонатология и друг раздел от ръководството по акушерство и гинекология. При другите два въпроса трите модела извличат релевантни откъси и отговорите са сравними.

Таблица 3: Позиция на откъса с отговора сред петте извлечени и възпроизведени факти по модели за векторно представяне (подсказка V2)

Модел	Хиперемезис	Инсулин	Дерматит
bge-m3	№1; 7/14	2/7	№1; 4/8
multilingual-e5-base	не е извлечен; 1/14	3/7	№1; 3/8
paraphrase-multilingual-mpnet-base-v2	не е извлечен; 1/14	2/7	№2; 4/8

Ограничение на дължината на входа. Причината е несъответствието, посочено в раздел 3.2, между дължината на откъсите и максималната дължина на входа на моделите (таблица 4). Откъсите са с медиана около 860 токена и максимум около 1410. `paraphrase-multilingual-mpnet-base-v2` представя средно едва 15% от всеки откъс, т.е. първите около 60 думи, а в откъса за хиперемезис терминът се появява едва около 440-ия токен, така че моделът никога не го „вижда“. `multilingual-e5-base` обхваща термина, но не и описанието на лечението след него. Освен това стойностите на сходство при него са сгъстени (0.84–0.85 за всичките пет извлечени откъса), така че и малки отклонения в представянето променят класирането. Само `BAAI/bge-m3` представя откъсите изцяло.

Таблица 4: Обхват на представянето на откъсите при максималната дължина на входа на всеки модел

Модел	Макс. вход (токени)	Отрязани откъси	Представена част	Позиция на „hyperemesis“
bge-m3	8192	0%	100%	438
multilingual-e5-base	512	99%	60%	439
paraphrase-multilingual-mpnet-base-v2	128	100%	15%	437

Следователно досегашното сравнение на трите модела не е равностойно, защото два от кандидатите се оценяват върху непълно представяне на откъсите. Размерът на откъсите е определен в преддипломния проект преди избора на модел за векторно представяне и за `paraphrase-multilingual-mpnet-base-v2` е неподходящ по конструкция.

Отворени проблеми за следващата итерация.

1. *Несвързани откъси.* Когато по дадена тема корпусът съдържа един релевантен откъс, фиксираният брой $k = 5$ добавя четири несвързани. При хиперемезис те са от

ръководствата по ендокринология, нефрология и гастроентерология и от друг раздел на ръководството по акушерство и гинекология, със сходство 0.59–0.60 срещу 0.65 за релевантния, така че праг, който да ги отдели, не е очевиден. След обръщането на реда те вече не влияят на отговора, но продължават да се показват като източници. Възможни решения са повторно класиране с отделен модел и праг на сходството, калибриран поотделно за всеки модел за векторно представяне, тъй като стойностите на сходство при трите модела са в различни диапазони.

2. *Размер на откъсите спрямо ограничението на моделите.* За равностойно сравнение откъсите трябва да бъдат съобразени с максималната дължина на входа, например чрез по-кратки подоткъси, чиито представяния се съпоставят с изходния откъс.
3. *Граници на откъсите.* Откъсът за хиперемезис завършва по средата на списъка с използвани лекарства, така че част от информацията остава в съседния откъс, който не е извлечен.
4. *Формална оценка.* Изводите в този раздел почиват на три въпроса и следва да бъдат проверени върху по-голям набор от ръчно анотирани въпроси (раздел 4.6).
5. *Ресурси.* Сървърът, чрез който Ollama изпълнява модела, пази в оперативната памет кеш на обработените подсказки с лимит 8 GB. При серия от дълги подсказки кешът достигна този лимит и изчерпа паметта на машината за разработка, така че зареждането на други модели стана невъзможно. Това е практическо ограничение за локалното изпълнение, което следва да се отчита при по-нататъшните експерименти.

4.3 Втора итерация на подобряване на системата

Втората итерация е насочена към отворените проблеми от първата, свързани с извличането: несвързаните откъси, които фиксираният брой k добавя към контекста, неравностойното сравнение на моделите за векторно представяне и липсата на количествена оценка върху повече от три въпроса. За разлика от първата итерация, в която се променяше подсказката при фиксирани откъси, тук се сравняват методи за извличане. Затова първо е съставен набор за разработка с автоматично изчислими показатели. Промяната в извличането разкри и нов проблем при генерирането, описан в края на раздела.

Набор за разработка. От всяко от 17-те ръководства са избрани случайно по един откъс, а от четирите ръководства с поне 100 откъса – по два, като изборът е ограничен до откъси, в които се споменават дози. Към тях са добавени трите откъса от първата итерация, общо 24. За всеки откъс е формулиран въпрос, на който той отговаря, и този откъс се счита за верния резултат от извличането. Наборът служи за сравнение на варианти по време на разработката, а не за окончателна оценка, и има две ограничения, които трябва да се имат предвид при тълкуването на резултатите. Първо, при 24 въпроса един въпрос съответства на 0.04 от всеки показател, така че разлики от един-два въпроса не са показателни. Второ, въпросите са формулирани при наличен текст на откъса и неизбежно повтарят част от думите му, което облагодетелства лексикалното търсене. Окончателната оценка върху независимо формулирани въпроси е предмет на раздел 4.6.

Показатели. Използват се два стандартни показателя за извличане. Пълнотата при k (Recall@k) е делът на въпросите, при които верният откъс е сред първите k извлечени. Пълнотата при 5 съответства на броя откъси, подавани на генеративния модел, а пълнота-

та при 20 – на броя кандидати за повторното класиране. Средният реципрочен ранг (Mean Reciprocal Rank) е средната стойност на $1/r$, където r е позицията на верния откъс, като се отчита нула, ако той не е сред първите десет. За разлика от пълнотата, този показател отчита и колко високо е класиран верният откъс. Оценката е строга: за верен се приема само конкретният откъс, въпреки че съседните откъси от същото ръководство понякога съдържат част от отговора. Тъй като всички методи се оценяват върху едни и същи въпроси, разликите между тях се проверяват за статистическа значимост със сдвоени тестове. За пълнотата, която за всеки въпрос е „да“ или „не“, се използва точният тест на Макнемар (McNemar's test) [26], който отчита само въпросите, при които двата метода се различават. За средния реципрочен ранг се използва рандомизационен тест (randomization test), препоръчан за оценка на извличането от Smucker и др. [27]. При него знакът на разликата за всеки въпрос се обръща случайно 100 000 пъти, а p-стойността е делът на случаите, в които така получената разлика в средното е поне толкова голяма, колкото наблюдаваната.

Таблица 5: Сравнение на методите за извличане върху набора за разработка (24 въпроса)

Метод	Пълнота при 1	Пълнота при 5	Пълнота при 20	Среден реципрочен ранг
Семантично търсене				
paraphrase-multilingual-mpnet-base-v2	0.04	0.25	0.46	0.15
multilingual-e5-base	0.42	0.62	0.75	0.50
embeddinggemma	0.33	0.71	0.83	0.48
qwen3-embedding (0.6 млрд. параметри)	0.46	0.83	0.96	0.61
bge-m3	0.54	0.79	0.92	0.65
Лексикално търсене				
BM25	0.25	0.79	0.92	0.44
BM25 със съкращаване на думите	0.42	0.79	0.92	0.55
Хибридно търсене				
сливане на bge-m3 и BM25	0.46	0.83	0.96	0.60
сливане и повторно класиране	**0.58**	**0.92**	**0.96**	**0.74**

Модели за векторно представяне. Таблица 5 потвърждава количествено наблюденията от първата итерация. `paraphrase-multilingual-mpnet-base-v2` е практически неизползваем: верният откъс е на първо място само при един от 24 въпроса и не е сред първите 20 при повече от половината. Това е единствената разлика в таблицата, която е статистически значима ($p < 0.001$). Следователно моделът не може да служи дори като първа стъпка, чиито грешки да се поправят от повторното класиране. `multilingual-e5-base` е по-добър, но отстъпва на `BAAI/bge-m3` по всички показатели, в съответствие с по-непълното представяне на откъсите (таблица 4). При 24 въпроса обаче и тази разлика не е значима. Като алтернативи на `BAAI/bge-m3` са изпробвани и два по-нови компактни многоезични модела, изпълнявани чрез Ollama, чиято максимална дължина на входа е достатъчна за откъсите: `qwen3-embedding` с 0.6 милиарда параметри [28] и `embeddinggemma` с 0.3 милиарда параметри [29]. За да е сравнението равностойно, `BAAI/bge-m3` е изпълнен и по същия начин и дава практически същите резултати като през библиотеката `sentence-transformers`. `qwen3-embedding` е наравно с `BAAI/bge-m3`: по-добър е по пълнота при 5 и при 20, по-слаб по пълнота при 1, като

разликите са от един-два въпроса. embeddinggemma е по-слаб, като разликата в средния реципрочен ранг е на границата на значимостта (0.48 срещу 0.65, p = 0.051). Нито един от двата не дава основание за смяна на модела. Семейството Qwen3 включва и модели за повторно класиране, които не са изпробвани в тази итерация.

Варианти на представянето. Изпробвани са и три начина за подобряване на представянето при BAAI/bge-m3: нормализиране на текста, добавяне на заглавието на ръководството и на раздела пред текста на откъса и разделяне на откъса на застъпващи се подоткъси от 120 или 200 думи, при което откъсът се класира по най-близкия си подоткъс. Последният вариант беше предложен в първата итерация като начин да се съобразят откъсите с ограничението на дължината на входа. Нито един от вариантите не подобрява последователно резултата: пълнотата при 1 е между 0.38 и 0.54, пълнотата при 5 – между 0.67 и 0.83, а средният реципрочен ранг – между 0.57 и 0.66, спрямо 0.54, 0.79 и 0.65 за изходния вариант. Разликите са в границите на един-два въпроса и в двете посоки, като нито една не е статистически значима (най-малката p-стойност е 0.25). Вероятното обяснение е, че BAAI/bge-m3 и без това обработва откъсите изцяло, а подоткъсите разделят сведения, които в заявката са свързани, например името на заболяването и описанието на лечението. Поради това разделянето е отхвърлено. За двата модела с кратък вход то не е изпробвано, защото при наличието на BAAI/bge-m3 подобряването им има само сравнителна стойност.

Лексикално търсене. BM25 без обработка на думите класира верния откъс на първо място едва при 6 от 24 въпроса. Причината е богатата морфология на българския език: „лечение“, „лечението“ и „лечения“ са различни думи за функцията за класиране. Съкращаването на всяка дума до първите шест знака, грубо приближение на основата ѝ (stemming), повишава пълнотата при 1 от 0.25 на 0.42 и средния реципрочен ранг от 0.44 на 0.55. Това е ясна тенденция, но не и доказано подобрение (p = 0.22 и 0.14). По пълнота при 5 и при 20 лексикалното търсене се изравнява с BAAI/bge-m3, въпреки че не използва никакъв обучен модел. Това отчасти се дължи на отбелязаното облагодетелстване от начина на съставяне на въпросите. По-важно е обаче, че двата метода грешат при различни въпроси: сред първите 20 резултата BAAI/bge-m3 намира верния откъс при два въпроса, при които лексикалното търсене не го намира, и обратно, а поне единият от тях го намира при всичките 24 въпроса. Дължината от шест знака не е настройвана систематично, а морфологичен анализатор би бил по-прецизна алтернатива.

Сливане и повторно класиране. Сливането по реципрочни рангове повишава пълнотата при 20 до 0.96, т.е. обединеният списък съдържа верния откъс при 23 от 24 въпроса. Сливането обаче понижава пълнотата при 1 спрямо BAAI/bge-m3 самостоятелно (0.46 срещу 0.54). Това е разлика от два въпроса, която не е значима (p = 0.63), но посоката ѝ е обяснима. То увеличава шанса верният откъс да е сред кандидатите, но не и да е на първо място, тъй като позицията му се усреднява с тази от по-слабия за дадения въпрос метод. Затова сливането е само първа стъпка, последвана от повторно класиране на първите 20 кандидата. Така се постигат най-добрите стойности по всички показатели: пълнота при 1 от 0.58, пълнота при 5 от 0.92 и среден реципрочен ранг от 0.74. Спрямо BAAI/bge-m3 самостоятелно най-голямо е подобрението при пълнотата при 5, която определя дали верният откъс изобщо достига до генеративния модел: от 19 на 22 въпроса. Горната граница на подхода е пълнотата на кандидатите: при единствения въпрос, чийто откъс не е сред тях, повторното класиране не може да помогне. Повторното класиране има и ограничение:

при въпроса за проследяването на бъбречната функция при хепатит B първите пет откъса са от други ръководства, с оценки от 0.78 до 0.98. Високата оценка показва, че откъсът е тематично близък до въпроса, но не гарантира, че е от вярното ръководство.

Статистическа значимост. Разликите между най-добрите методи са от порядъка на няколко въпроса и при 24 въпроса не са статистически значими (таблица 6). Хибридното извличане с повторно класиране превъзхожда `BAAI/bge-m3` по пълнота при 5 при три въпроса и не отстъпва при нито един, но вероятността такова разпределение да се получи случайно е 0.25. По пълнота при 1 повторното класиране печели при пет въпроса и губи при четири. Най-близо до значимост е подобрението на средния реципрочен ранг от повторното класиране спрямо самото сливане ($p = 0.095$). Следователно резултатите от тази итерация показват последователна посока: никой от показателите на окончателния метод не е по-лош от тези на `BAAI/bge-m3`. Те обаче не доказват подобрение и трябва да се потвърдят върху по-голям набор. Ако съотношението между спечелените и загубените въпроси се запази, за да се открие подобрението на пълнотата при 5 с мощност 80% при ниво на значимост 0.05, са нужни около 60 въпроса. Ако повторното класиране губи и при един от 24 въпроса, са нужни около 190 [30]. Това определя минималния размер на набора за окончателната оценка (раздел 4.6). Всички тестове се изчисляват автоматично (`evaluation/significance.py`) върху запазените класирания и отговори от експериментите, така че резултатът не зависи от неповторимостта на генерирането, обсъдена по-долу.

Таблица 6: Значимост на основните разлики при извличането (24 въпроса). Посочени са въпросите, при които само единият метод е успешен, и p-стойността на сдвоения тест.

Сравнение (А → Б)	Пълнота при 1: само А / само Б, p	Пълнота при 5: само А / само Б, p	Среден реципрочен ранг: А → Б, p
bge-m3 → сливане и повторно класиране	4 / 5; 1.00	0 / 3; 0.25	0.65 → 0.74; 0.27
сливане → сливане и повторно класиране	3 / 6; 0.51	0 / 2; 0.50	0.60 → 0.74; 0.095
BM25 → BM25 със съкращаване	1 / 5; 0.22	1 / 1; 1.00	0.44 → 0.55; 0.14
bge-m3 → multilingual-e5-base	6 / 3; 0.51	5 / 1; 0.22	0.65 → 0.50; 0.15
bge-m3 → qwen3-embedding	4 / 2; 0.69	1 / 2; 1.00	0.65 → 0.61; 0.67
bge-m3 → paraphrase-multilingual-mpnet-base-v2	12 / 0; <0.001	13 / 0; <0.001	0.65 → 0.15; <0.001

Праг за отказ. Оценката на модела за повторно класиране е в една и съща скала за всички заявки, за разлика от косинусовата близост, чиито стойности зависят от модела за векторно представяне и при `multilingual-e5-base` са сгъстени в тесен интервал. Затова прагът е определен върху нея. За калибрирането освен 24-те въпроса от набора за разработка са използвани десет въпроса извън обхвата на корпуса. Седем от тях са общи („Колко е висок връх Мусала?“, „Как се приготвя баница?“), а три са близки до медицината, но не и до фармакотерапевтичните ръководства („Как се лекува счупен крак?“, „Как се поставя пломба на зъб?“, „Каква е дозата на парацетамол за куче?“). Най-високата оценка при въпросите извън обхвата е 0.049, при въпроса за парацетамола, а при останалите

е под 0.025. Най-ниската оценка на първия откъс при въпросите от набора е 0.717. Прагът 0.1 е избран в интервала между двете стойности, с двукратен запас над най-високата оценка извън обхвата. Така системата отказва да отговори на всичките десет въпроса извън обхвата, без да вика генеративния модел. Прагът има и втори ефект, който решава първия отворен проблем от първата итерация. Прилага се към всеки откъс поотделно, затова отпадат несвързаните откъси, които фиксираният брой k добавяше. При въпроса за хиперемезис остава само релевантният откъс, с оценка 0.98, докато останалите кандидати получават най-много 0.02, при косинусова близост 0.59–0.60 срещу 0.65. От 24-те въпроса 21 запазват пет откъса, а три – по-малко (един, два и три), като верният откъс не е отпаднал при нито един въпрос. Изводът за прага почива на малък брой въпроси извън обхвата, повечето от които са далеч от темата на корпуса. Най-рискови са медицински въпроси по теми, които ръководствата не покриват, но с близък речник, и прагът трябва да бъде проверен върху такива въпроси при окончателната оценка.

Изчислителна цена. На централния процесор повторното класиране на 20 кандидата отнема около 70 секунди на въпрос, което е неприемливо за интерактивна система. Машината за разработка разполага с графичен процесор NVIDIA GeForce RTX 2060 с 6 GB памет, който дотогава се използваше само от Ollama за генеративния модел, докато библиотеката PyTorch, чрез която се изпълняват моделите за векторно представяне и повторно класиране, беше във вариант само за централен процесор. След преминаването към варианта ѝ за графичен процесор (CUDA) моделите се изпълняват в половинна точност (fp16), т.е. с 16-битови числа с плаваща запетая вместо 32-битови. Това намалява двойно заеманата памет и е необходимо, за да се поберат всички модели в 6 GB. Таблица 7 обобщава резултата. Половинната точност не променя резултатите: подредбата след повторното класиране съвпада изцяло с получената на централния процесор, а векторните представяния на откъсите имат косинусова близост поне 0.9997 с изчислените преди това. Затова вече построените индекси са запазени. При заредени три модела – за векторно представяне, за повторно класиране и генеративен – заетата видеопамет е около 5.6 от 6 GB, заедно с тази на операционната система, като генеративният модел остава изцяло на графичния процесор.

Таблица 7: Време за изпълнение на централния и на графичния процесор

Задача	Централен процесор	Графичен процесор, половинна точност
Повторно класиране на 20 кандидата за един въпрос	около 70 с	0.97 с
Индексиране на корпуса с `bge-m3`	116 мин	84 с
Цялото извличане за един въпрос	около 70 с	около 1 с

Зацикляне при кратък контекст. Прагът за релевантност разкри нов проблем при генерирането. При въпроса за хиперемезис до модела вече достига само един откъс, в който групите лекарства са изброени заедно с АТС кодовете им. Моделът започна да повтаря едни и същи четири реда, докато не изчерпа контекстния прозорец, което в приложението изглежда като блокиране за около две минути. При температура нула всеки следващ токен се избира като най-вероятния (алчно декодиране, greedy decoding), а този начин на декодиране е известен със склонността си към повторения, тъй като всяко повторение прави

следващото още по-вероятно [31]. Вероятно ефектът се засилва от списъчната структура на единствения откъс, чийто шаблон моделът продължава.

Разгледани са четири възможни мерки. Връщането към случаен избор с температура над нула е отхвърлено, защото в първата итерация доведе до различни отговори на един и същ въпрос, единият с невярно твърдение. Ограничението на дължината на отговора спира зацикляне, но не и безсмисления текст преди това. Остават наказанието за повторение, което намалява вероятността на токени, срещнали се сред последните обработени, и разширяването на прозореца, в който се отчитат повторенията. Предварителен опит върху шест въпроса показа, че разширяването на прозореца до 512 токена без посилно наказание не спира зацикляне. Затова по-подробно е сравнена силата на наказанието, върху всичките 24 въпроса от набора за разработка и с откъсите от хибридното извличане (таблица 8).

Освен зацикляния и възпроизведените факти, таблицата отчита и нова мярка: числата и ATC кодовете в отговора, които не се срещат в подадените откъси. Числата и кодовете са най-опасният вид неподкрепена подробност във фармакотерапията, защото изразяват дози, прагове и идентификатори на лекарства. За разлика от броенето на факти, проверката не изисква предварително изброени факти и затова се прилага към всички 24 въпроса. Преди сравнението латинските и кирилските букви с еднакъв вид се уеднаквяват, тъй като при разпознаването на текста от PDF документите те се смесват в кодовете: кодът на ranitidine в откъса е „A02BA02“ с кирилски „А“ и „В“. Номерата на точките в изброяванията в отговора се пропускат.

Таблица 8: Влияние на наказанието за повторение (24 въпроса, откъси от хибридното извличане, температура нула)

Наказание	Зацикляния	Факти (6 въпроса)	Числа и кодове	Отсъстващи от откъсите	Средна дължина (токени)
няма (1.0)	1	21/44	67	1	177
1.05	0	24/44	75	6	130
1.1	0	17/44	60	1	125
1.2	0	25/44	51	2	149

Избор на наказанието. Всяка от изпробваните стойности премахва зацикляне. Стойността 1.2 дава най-много възпроизведени факти, а при предварителния опит отговорът за хиперемезис съдържа 12 от 14 факта срещу 7 за варианта V2 от първата итерация, включително хоспитализацията и покоя, които тогава бяха пропуснати. Разликата спрямо алчното декодиране (25 срещу 21 от 44 факта) не е статистически значима ($p = 0.34$ при сдвояване по факт). Тъй като фактите от един отговор не са независими, тази стойност по-скоро подценява несигурността. Решаващо за избора е премахването на зацикляне, което прави отговора неизползваем. Числата и кодовете, отсъстващи от откъсите, са малко при всички стойности и не нарастват систематично със силата на наказанието. Ръчната проверка на отбелязаните случаи показва, че мярката открива истински грешки. Сред тях са ATC кодове, приписани на грешни лекарства (без наказание и при 1.05), измислената схема на прилагане „на ден 1, 4, 7, …, 31“ (при 1.05) и доза на ganciclovir от „8–16 mg/kg“, каквато в откъсите няма (при 1.2). Два от случаите обаче са фалшиви сигнали, дължащи се на различен запис на едно и също число: „90,000“ в отговора срещу „90 000“ в откъса. Прието

е наказание 1.2 заедно с ограничение на отговора до 1024 токена, т.е. около 12 секунди. То не е нужно при нормална работа, където отговорите са средно около 150 токена, но ограничава времето за отговор, ако зацикляването все пак се появи. Прекъснатият отговор завършва с „[…]“, за да не бъде приет за пълен.

Повторимост. Сравнението на два прогона с едни и същи въпроси показа, че температура нула не гарантира напълно еднакви отговори, както беше прието в първата итерация. При многократно подаване на една и съща подсказка се получават до два различни отговора, които съвпадат в началото и се разклоняват на различно място, в наблюдаваните случаи между 16-ия и 453-ия знак. Това се наблюдава и без наказание за повторение, а фиксирането на началната стойност на генератора на случайни числа (seed) не го променя. Следователно разликите не се дължат на случаен избор. Вероятната причина са малки числени разлики в изчисленията на графичния процесор, например в зависимост от това дали подсказката вече е обработена и запазена в кеша на Ollama. При два почти еднакво вероятни токена такава разлика променя избора, а оттам и целия следващ текст. На практика отговорите остават близки, но разлики между вариантите от порядъка на един-два факта не са показателни. Например наказание 1.2 дава 25 и 26 от 44 факта в два прогона с едни и същи откъси.

Генеративен модел. Като алтернатива на BgGPT с 2.6 милиарда параметри е изпробван по-новият вариант, изграден върху Gemma 3, с 4 милиарда параметри (`INSAIT-Institute/BgGPT-Gemma-3-4B-IT`), също с 4-битово квантуване. Той се зарежда изцяло на графичния процесор (2.8 GB) и генерира 66 токена в секунда срещу 83 за по-малкия модел. При шестте въпроса и откъсите от `BAAI/bge-m3`, преди въвеждането на повторното класиране и на наказанието за повторение, той възпроизвежда 32 от 44 факта срещу 22 за по-малкия модел. Ръчният прочит обаче показва, че по-дългите му отговори съдържат и повече неподкрепени подробности. При хиперемезис той приписва на метоклопрамида, прометазина и атропина ATC кодове, каквито в откъсите няма. При еклампсията изрежда антихипертензивни лекарства, сред които metoprolol, който не се споменава в откъсите, и пропуска посочения в тях methyldopa. Както при конкретизиращата инструкция в първата итерация, по-високият брой възпроизведени факти не означава по-вярен отговор. Затова по-големият модел засега не е приет. Изборът ще бъде преразгледан след въвеждането на автоматична проверка за неподкрепени подробности, която би позволила да се използва по-голямата му пълнота, като грешките му се откриват. Допълнително ограничение е видеопаметта: той заема с 0.7 GB повече от сегашния модел, а с трите модела вече е заета почти цялата памет на графичния процесор.

Резултат. Стойностите на всички показатели за извличането нарастват спрямо `BAAI/bge-m3` самостоятелно: пълнотата при 1 от 0.54 на 0.58, пълнотата при 5 – от 0.79 на 0.92, а средният реципрочен ранг – от 0.65 на 0.74. При 24 въпроса тези разлики не са статистически значими и трябва да се потвърдят при окончателната оценка. Качествените промени обаче не зависят от размера на набора. Несвързаните откъси вече не достигат до генеративния модел и не се показват като източници. Системата отказва да отговори на въпроси извън обхвата на корпуса, без да вика генеративния модел, а цялото извличане отнема около секунда. Така се изпълнява и препоръката на Li и Zhu [11] за изрично съобщение при липса на релевантен откъс, която в първата итерация остана неизпълнена. Генерирането вече не зацикля, а отговорът за хиперемезис съдържа по-голямата част от

сведенията в откъса.

Отворени проблеми за следващата итерация.

1. *Неподкрепени подробности.* Моделът продължава да допуска грешни АТС кодове и дози, включително когато те са в откъса. Проверката на числата и кодовете от този раздел може да се превърне в проверка, прилагана автоматично към всеки отговор, като се разшири с имената на лекарствата и с уеднаквяване на записа на числата.
2. *Граници на откъсите.* Откъсът за хиперемезис продължава да завършва по средата на списъка с използвани лекарства. Възможни решения са добавянето на съседните откъси към контекста или фрагментиране по раздели на ръководствата.
3. *Проверка на прага.* Прагът за отказ трябва да бъде проверен върху медицински въпроси по теми, които ръководствата не покриват (раздел 4.6).
4. *Повторимост.* Сравненията на варианти на генерирането трябва да почиват на няколко прогона или на разлики, по-големи от тези между прогоните.

4.4 Трета итерация: проверка за неподкрепени подробности

Грешните АТС кодове, дози и имена на лекарства, отбелязани като първи отворен проблем във втората итерация, не могат да бъдат отстранени надеждно чрез подсказката, тъй като ги допускат и двата изпробвани генеративни модела. Затова всеки отговор се проверява автоматично спрямо показаните откъси (`src/medguiderag/generation/grounding.py`). Алтернативата – оценка на верността на целия отговор от друг езиков модел в ролята на съдия [5] – би изисквала втори модел в почти изчерпаната видеопамет и би удължила отговора. Затова проверката е детерминистична и се ограничава до три вида подробности, които във фармакотерапията носят най-голям риск и могат да бъдат проверени дословно:

- имена на лекарства, които не се срещат в откъсите;
- АТС кодове, които липсват в откъсите или в тях стоят до друго лекарство;
- числа (дози, прагове, срокове), които не се срещат в откъсите.

Отбелязаните подробности се оцветяват в отговора и се изброяват в предупреждение под него, но самият отговор не се променя. Преценката остава за потребителя, който вижда и откъсите.

Основната трудност е, че една и съща подробност се записва различно в отговора и в откъсите. Ръководствата дават лекарствата с международните им непатентни наименования (INN) на латиница, а моделът често ги пише на кирилица, затова двете форми се свеждат до обща транслитерация („метоклопрамид“ и „Metoclopramide“). Кирилска дума се приема за име на лекарство, ако латинската ѝ форма се среща в корпуса поне толкова често, колкото кирилската. Иначе общи думи като „терапия“ биха били отбелязвани, защото „therapy“ също се среща в корпуса. Кодовете в откъсите съдържат кирилски букви и погрешно разпознати цифри („A02BAO3“ вместо A02BA03), а числата се записват с различни разделители („90 000“ и „90,000“), затова и двете се уеднаквяват преди сравнението. Имената се сравняват с допуск от една или две букви, за да не се отбелязват правописни варианти („хидрохлортиазид“ спрямо hydrochlorothiazide), а номерата на точките в изброяванията, числата от самия въпрос и позоваванията на страници не се проверяват.

Оценка. Проверката е приложена към всички 132 отговора от експериментите във втората итерация. Тя отбелязва 44 подробности в 15 отговора, от които ръчният прочит потвърждава 41 като истински грешки, т.е. 93% с 95% доверителен интервал на Уилсън (Wilson score interval) от 82 до 98% [32]. Сред тях са всички грешки, описани във втората итерация: АТС кодовете, приписани на грешни лекарства, измислената схема „на ден 1, 4, 7, …, 31“, дозата на ganciclovir, както и кодовете и metoprolol в отговорите на модела с 4 милиарда параметри. Отбелязани са и несъществуващи имена на лекарства („Chlormizine“, „Bromodies“) и четири грешки, пропуснати при ръчния прочит: левотироксин и парацетамол, добавени без опора в откъсите, и два кода, каквито в откъсите няма. Трите фалшиви сигнала се дължат на превод на термин от откъса („Sodium“ за „натриев“). Пълнотата на проверката не е измерена, защото това изисква ръчна анотация на всички отговори.

Ограничения. Проверката установява дали дадена подробност се среща в откъсите, но не и дали е свързана с правилното твърдение. Доза, отнесена към друг ден от схемата, вярно име с погрешно показание и пропуснатите сведения остават неоткрити. Затова тя допълва, но не замества проверката на източниците от потребителя.

Преразглеждане на генеративния модел. С въведената проверка моделът с 4 милиарда параметри е сравнен отново с досегашния, при сегашните настройки на системата и в два прогона върху 24-те въпроса. Двата прогона дадоха еднакви резултати. По-големият модел възпроизвежда 29 срещу 25 от 44 факта, а проверката отбелязва неподкрепена подробност в един негов отговор (доза на daunorubicine в блока HR2) срещу два за по-малкия модел. Нито една от двете разлики не е статистически значима: при фактите $p = 0.45$. Затова изборът почива главно на ръчния прочит. Ръчният прочит на отговорите на шестте въпроса и на двата най-сложни (блокът HR2 и неоадювантната терапия при HER2-позитивен карцином) показва по-пълни отговори при по-големия модел. Такива са пълната схема на магнезиевия сулфат и диазепама при еклампсия и локалната терапия при атопичен дерматит. Показва обаче и грешки, които проверката не открива: дози и дни, отнесени към грешно лекарство в блока HR2, мерна единица mg/kg вместо mg/m² при paclitaxel и целеви стойности, представени като показания за инсулин. Сходни грешки допуска и по-малкият модел, в блока HR2 и при антибиотиците за атопичния дерматит. Затова по подразбиране е приет по-големият модел, а по-малкият остава избираем от интерфейса. Трите модела заемат заедно около 5.7 от 6 GB видеопамет, генеративният остава изцяло на графичния процесор, а отговорът отнема средно около 7 секунди срещу около 5 за по-малкия модел.

4.5 Четвърта итерация: маркиране на използваните части от източниците

Системата показва до пет откъса от по около 420 думи, така че проверката на един отговор изисква прочитането на около 2000 думи. В раздел 3.4 проследимостта беше постигната на ниво откъс, като съзнателно отклонение от препоръката на Li и Zhu [11] за източник към всяко твърдение. Четвъртата итерация стеснява проследимостта до изречение: в показаните откъси се маркират изреченията, на които стъпва отговорът (`src/medguiderag/generation/attribution.py`).

Разгледани са три алтернативи. Генеративният модел да посочва сам източниците е отхвърлено, защото в първата итерация той измисляше цитати. Модел за логическо следване (natural language inference) или кръстосан енкодер върху всички двойки изречения

би бил по-точен, но би добавил трети модел в почти изчерпаната видеопамет или стотици оценки на отговор. Маркирането направо в оригиналните PDF документи изисква откриване на текста по страниците, което разпознаването на текста от сканираните документи затруднява, и е оставено за бъдещо развитие. Избраният подход съпоставя изреченията с два признака, които вече са налични в системата. Семантичният е косинусовата близост между векторните представяния, изчислени с модела за векторно представяне, който е зареден за извличането. Лексикалният е делът от имената, числата и думите на изречението от отговора, които се срещат в изречението от откъса. Използва се нормализирането от третата итерация, а всяка дума тежи толкова повече, колкото по-рядко се среща в откъсите, така че общи думи като „лечение“ почти не влияят. Оценката на двойката е средното от двата признака. Изречение от отговора се свързва с изречение от откъс при оценка поне 0.45, като се допускат до три връзки, ако отговорът съчетава няколко факта. Въвеждащите изречения от рода на „Лечението включва:“ не се съпоставят. В интерфейса свързаните изречения се оцветяват, всеки откъс е обозначен с броя маркирани изречения или като „не е използван в отговора“, а изреченията от отговора без съответствие се изброяват отделно. Съпоставянето не удължава забележимо отговора.

Оценка. Прагът е определен чрез ръчен преглед на най-добрите съвпадения за изреченията в отговорите на модела с 4 милиарда параметри на 24-те въпроса. Под 0.45 съвпаденията бяха предимно грешни или засягаха въвеждащи изречения. При окончателните настройки от 93 изречения в тези отговори 87 са свързани с общо 136 изречения от откъсите, а 6 остават без съответствие. Използвани са 49 от 111-те показани откъса. Ръчният преглед на всички 136 връзки показва, че 118 (87%, 95% доверителен интервал от 80 до 91%) сочат вярното място, 11 сочат сходен текст на друго място и 7 не могат да бъдат оценени еднозначно, защото таблица от документа е разпозната като едно дълго изречение. Типичната грешка е сходен текст в съседен раздел, например режимът за късен вместо за ранен сифилис или редът на друг блок от таблицата на протокола HR2. От шестте изречения без съответствие две наистина нямат опора в откъсите, три са пропуснати съвпадения (неправилно изписани имена на лекарства, перифраза и изречение с два факта), а едно е спорно. Тъй като прагът е определен и оценен върху едни и същи отговори, резултатът е по-скоро горна граница.

Полза. Съпоставянето допълва проверката от третата итерация. При въпроса за първично рефрактерната остра миелоидна левкемия откъсът с отговора не е извлечен и моделът отговаря от собствената си памет. Проверката за неподкрепени подробности не отбелязва нищо, защото отговорът не съдържа числа и имена на лекарства, но съпоставянето не намира опора за изречението и обозначава и петте откъса като неизползвани. Освен това погрешните твърдения често се свързват точно с мястото, което ги опровергава. Така „Инфузиите се прекратяват след раждането“ сочи изречението „Инфузията се прекратява 24 часа след раждането“, а дозата на paclitaxel в mg/kg сочи реда със 175 mg/m², което улеснява откриването на грешката.

Ограничения. Маркираното изречение показва откъде идва дадено твърдение, но не и че то е предадено вярно. Точността на маркирането зависи от разделянето на текста на изречения, което при таблиците и списъците в документите е грубо. Прагът е определен върху малък набор и трябва да бъде проверен при окончателната оценка (раздел 4.6).

4.6 Качествен анализ и оценка

5 Техническа реализация

Тази глава описва как е реализирана системата: използваните инструменти, организацията на кода и данните и средствата за проверка и възпроизвеждане на резултатите. Изборът на методите е обоснован в глава 3 и в итерациите от глава 4, затова тук се препраща към тях. Инсталирането и стартирането са описани в приложение 1.

5.1 Използвани инструменти и технологии

Системата е написана на Python, на който са и всички използвани библиотеки за работа с езикови модели, така че целият конвейер, от индексирането до уеб интерфейса, е на един език. Таблица 9 изброява инструментите с версиите, с които са получени резултатите в тази работа.

Таблица 9: Използвани инструменти и модели

Компонент	Инструмент (версия)	Роля
Език	Python 3.11.9	целият конвейер
Изчисления	PyTorch 2.14.0 (CUDA 12.6)	изпълнение на моделите за векторно представяне и повторно класиране на графичния процесор
Векторно представяне	sentence-transformers 6.0.1 (transformers 5.17.0)	зареждане на моделите, изчисляване на векторите, кръстосан енкодер
Векторна база	FAISS 1.15.0	точно търсене по косинусова близост (раздел 3.2)
Лексикално търсене	rank_bm25 0.2.2	функцията за класиране BM25
Генериране	Ollama 0.34.4 (llama.cpp), клиент ollama 0.6.2	локален сървър за генеративния модел
Уеб интерфейс	Streamlit 1.64.0	демонстрационното приложение (раздел 4.1)
Проверка на интерфейса	Playwright 1.63.0, Google Chrome	автоматизирано задаване на въпроси и екранни снимки
Модели		
Векторно представяне	`BAAI/bge-m3` (по подразбиране), `multilingual-e5-base`, `paraphrase-multilingual-mpnet-base-v2`	семантично търсене; съпоставяне на изреченията (раздел 4.5)
Повторно класиране	`BAAI/bge-reranker-v2-m3`	подреждане на кандидатите и праг за отказ
Генериране	BgGPT 4B (Gemma 3) и 2.6B (Gemma 2), 4-битово квантуване (Q4_K_M)	генериране на отговора

Повечето избори вече са обосновани в предходните глави: FAISS в раздел 3.2, BgGPT в раздели 3.4 и 4.4, Streamlit в раздел 4.1, а изчисленията на графичния проце-

сор в половинна точност в раздел 4.3. Остават два избора, които се отнасят само до реализацията.

Изпълнение на генеративния модел. Освен Ollama са разгледани три възможности за локално изпълнение на BgGPT. Библиотеката transformers зарежда теглата на модела в половинна точност, при което вариантът с 4 милиарда параметри заема около 8 GB, повече от наличните 6 GB видеопамет. Намаляването на паметта до 4 бита би изисквало допълнителни библиотеки за квантуване и не би използвало вече публикуваните от INSAIT квантувани версии. Библиотеката llama-cpp-python използва същия изпълнителен механизъм като Ollama (llama.cpp), но за работа на графичния процесор се компилира при инсталирането, което под Windows изисква отделни средства за компилиране и поддръжка на CUDA и прави инсталирането крехко. Сървърът vLLM е предназначен за едновременна работа на много потребители и поддържа официално само Linux. Ollama предлага готови изпълними файлове с поддръжка на CUDA, работи с квантуваните модели във формата GGUF и сам управлява зареждането и освобождаването им, а към него се изпращат заявки по протокола HTTP на локалния адрес. Цената на това удобство е по-малкият контрол: кешът на обработените подсказки в оперативната памет, достигащ до 8 GB, не може да бъде ограничен, което на машината за разработка неколкократно изчерпа паметта при серии от заявки.

Векторни представяния. Библиотеката sentence-transformers е предпочетена пред директното използване на transformers, защото предоставя общ интерфейс за моделите за векторно представяне и за кръстосания енкодер, с групова обработка и нормализиране на векторите. При директно използване на transformers обединяването на векторите на отделните токени във вектор на целия текст трябва да се реализира отделно за всеки модел, тъй като `BAAI/bge-m3` използва вектора на първия токен, а другите два модела – средното на всички. Грешка в тази стъпка би влошила извличането, без да е очевидна.

Машина за разработка. Системата е разработена и оценена на компютър с Windows 11, графичен процесор NVIDIA GeForce RTX 2060 с 6 GB видеопамет и 16 GB оперативна памет, от които при работа обикновено са свободни между 2 и 5 GB. Тези ограничения определиха няколко решения: 4-битовото квантуване на генеративния модел, половинната точност на останалите модели, зареждането на моделите веднъж и преизползването им при следващите въпроси и наличието на само един генеративен модел във видеопаметта. Определиха и начина на провеждане на експериментите: скриптовете за оценка се изпълняват при спряно приложение и периодично освобождават модела в Ollama, за да не се изчерпи паметта.

5.2 Структура на проекта и файлове

Кодът на системата е в пакета `src/medguiderag/` и е разделен на модули по стъпките на конвейера (таблица 10). Модулите са общо около 720 реда код, от които най-обемни са проверката за неподкрепени подробности и съпоставянето на изреченията. Освен пакета проектът съдържа папката `evaluation/` със скриптовете за оценка (раздел 5.3), папката `data/` с корпуса и индексите и папката `docs/` с текста на дипломната работа.

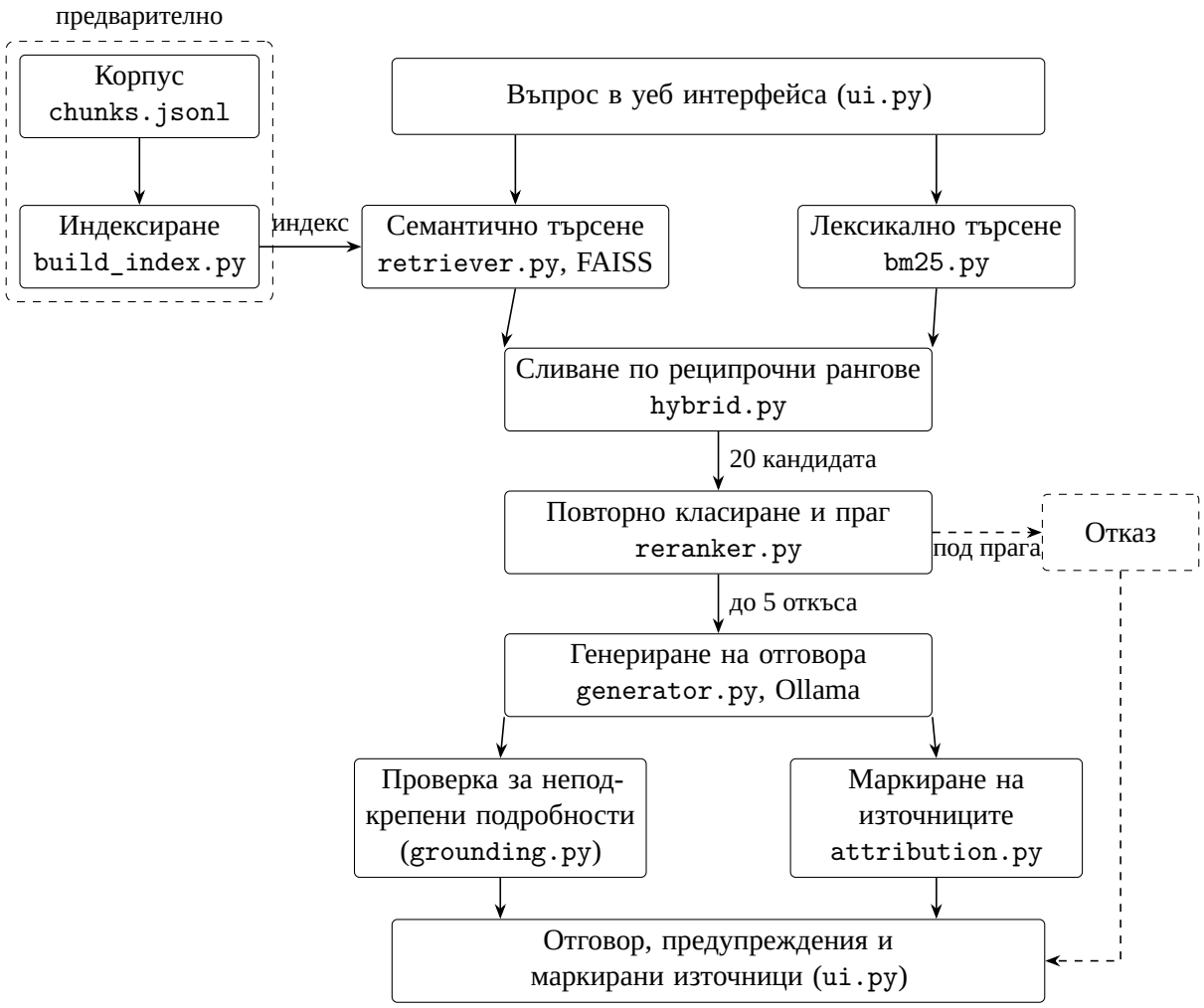
Таблица 10: Модули на пакета `src/medguiderag`

Файл	Предназначение
`config.py`	пътища до данните, модели, праг за отказ, брой кандидати
`ingest/load_chunks.py`	зареждане на откъсите от корпуса
`indexing/embed.py`	зареждане на модела за векторно представяне и изчисляване на нормализираните вектори
`indexing/build_index.py`	изграждане на индекса и метаданните за даден модел (от командния ред)
`retrieval/retriever.py`	семантично търсене в индекса
`retrieval/bm25.py`	лексикално търсене със съкращаване на думите
`retrieval/reranker.py`	модел за повторно класиране
`retrieval/hybrid.py`	хибридно извличане: сливане, повторно класиране, праг
`generation/generator.py`	подсказка и заявка към Ollama
`generation/grounding.py`	проверка за неподкрепени подробности
`generation/attribution.py`	съпоставяне на изреченията от отговора с откъсите
`app/ui.py`	уеб интерфейс

Фигура 4 показва как модулите се свързват при обработката на въпрос. Индексирането е предварителна стъпка, която се изпълнява веднъж за всеки модел за векторно представяне. Индексът за BM25 не се записва, а се изгражда в паметта при създаването на компонента за извличане, тъй като за 1380 откъса това отнема под секунда. Моделите се зареждат при първото използване и остават в паметта: на ниво модул чрез кеширане на функциите, които ги зареждат, и на ниво приложение чрез кеширане на ресурсите в Streamlit. Един и същ модел за векторно представяне служи и за извличането, и за съпоставянето на изреченията, а речникът с имената на лекарства, който използва проверката, се изгражда веднъж от целия корпус.

Данни. Корпусът е копие на резултата от преддипломния проект в папката `data/processed/`, така че системата не зависи от неговите файлове. Файлът `chunks.jsonl` съдържа по един ред за всеки от 1380-те откъса с текста, документа, началната и крайната страница, раздела, идентификатора и броя думи. Файлът `documents.jsonl` съдържа текста по страници (1872 страници). Системата не го използва при отговор, но той би позволил маркирането на източниците да се пренесе в оригиналните документи. За всеки модел за векторно представяне индексът е в отделна папка `data/index/`, наименувана по модела, и се състои от три файла: индекса на FAISS, метаданните на откъсите в същия ред като векторите и статистика за изграждането (модел, брой откъси, размерност, време).

Конфигурация. Параметрите, които определят поведението на системата, са събрани на две места. В `config.py` са генеративните модели с имената им в интерфейса, кандидатите за модел за векторно представяне, моделът за повторно класиране, броят кандидати за него (20) и прагът за отказ (0.1). В `generation/generator.py` са параметрите на генерирането: контекстен прозорец от 8192 токена, температура нула, наказание за повторение 1.2 и най-много 1024 генерирани токена. Размерът на шрифта в интерфейса се задава във файла с настройки на Streamlit. Така смяната на модел или на праг изисква промяна на един ред.



Фигура 4: Компоненти на системата и обработката на един въпрос. Пунктирът означава предварителната стъпка и пътя при отказ.

5.3 Проверка и възпроизводимост

За модулите няма автоматизирани тестове. Това е ограничение на реализацията, особено за нормализирането на имената, кодовете и числата в проверката за неподкрепени подробности и в съпоставянето на изреченията, където бяха нужни няколко поправки. Коректността е проверявана по два други начина.

Проверка на работещото приложение. След всяка промяна приложението се стартира и се проверява чрез скрипт, който с библиотеката Playwright управлява браузъра Google Chrome: задава въпрос, изчаква края на обработката, извежда отговора, предупрежденията и източниците и записва екранни снимки. Така се проверява системата такава, каквато я вижда потребителят, включително отказът и маркирането на източниците.

Възпроизвеждане на резултатите. Всяка таблица и всеки числов резултат от глава 4 се възпроизвеждат със самостоятелен скрипт от папката `evaluation/`, например `python -m evaluation.retrieval_eval` за сравнението на методите за извличане. Наборът за разработка, въпросите извън обхвата и ключовите факти са в `evaluation/data/`. Тъй като

генерирането на графичния процесор не е напълно повторимо (раздел 4.3), отговорите, класиранията и ръчните оценки, върху които са изчислени числата в текста, са запазени в `evaluation/results/thesis/`. Скриптовете, които не генерират нови отговори, като проверката за значимост и оценките на проверката и на маркирането, четат тези файлове и дават точно числата от текста. Скриптовете, които генерират отговори наново, дават същите изводи, но броят на възпроизведените факти може да се различава с един-два. Повторното пускане на всички скриптове потвърди числата в таблиците, като единствената разлика беше един факт при един от вариантите на подсказката в първата итерация. Изключение правят времената за изпълнение, които зависят от натоварването на машината: при повторното измерване повторното класиране отне 1.7 вместо 1.0 секунди на въпрос.

6 Изводи и препоръки за бъдещо развитие

7 Заключение

Литература

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, D. Kiela. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 2020. arXiv:2005.11401.

[2] S. Wu, Y. Xiong, Y. Cui, H. Wu, C. Chen, Y. Yuan, L. Huang, X. Liu, T.-W. Kuo, N. Guan, C. J. Xue. *Retrieval-Augmented Generation for Natural Language Processing: A Survey*. arXiv:2407.13193, 2024.

[3] C. Sharma. *Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers*. arXiv:2506.00054, 2025.

[4] A. Singh, A. Ehtesham, S. Kumar, T. Talaei Khoei, A. V. Vasilakos. *Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG*. arXiv:2501.09136, 2025.

[5] B. Malin, T. Kalganova, N. Boulgouris. *A Review of Faithfulness Metrics for Hallucination Assessment in Large Language Models*. arXiv:2501.00269, 2024.

[6] M. S. Tamber, F. S. Bao, C. Xu, G. Luo, S. Kazi, M. Bae, M. Li, O. Mendelevitch, R. Qu, J. Lin. *Benchmarking LLM Faithfulness in RAG with Evolving Leaderboards*. arXiv:2505.04847, 2025.

[7] K. Luo, M. Qin, Z. Liu, S. Xiao, J. Zhao, K. Liu. *Large Language Models as Foundations for Next-Gen Dense Retrieval: A Comprehensive Empirical Assessment*. arXiv:2408.12194, 2024.

[8] A. Rashmiks, T. Madushanka. *A Comprehensive Empirical Evaluation of Vector Database Systems for Approximate Nearest Neighbor Search: Performance, Quality, and Resource Trade-offs*. arXiv:2608.12812, 2026.

[9] J. He, B. Zhang, H. Rouhizadeh, Y. Chen, R. Yang, J. Lu, X. Chen, N. Liu, D. Teodoro. *Retrieval-Augmented Generation in Biomedicine: A Survey of Technologies, Datasets, and Clinical Applications*. arXiv:2505.01146, 2025.

[10] Y. Miao, Y. Zhao, Y. Luo, H. Wang, Y. Wu. *Improving Large Language Model Applications in the Medical and Nursing Domains With Retrieval-Augmented Generation: Scoping Review*. Journal of Medical Internet Research, 27:e80557, 2025.

[11] L. Li, W. Zhu. *Pursuing Best Industrial Practices for Retrieval-Augmented Generation in the Medical Domain*. arXiv:2602.03368, 2026.

[12] L. Ranaldi, B. Haddow, A. Birch. *Multilingual Retrieval-Augmented Generation for Knowledge-Intensive Task*. arXiv:2504.03616, 2025.

[13] I. Marinova, K. Simov, P. Osenova. *Transformer-Based Language Models for Bulgarian*. Proceedings of RANLP 2023 (Recent Advances in Natural Language Processing), 2023.

[14] N. Paev, K. Simov, P. Osenova, S. Petrova. *Large Language Models for Bulgarian*. CLaDA-BG Conference, 2024.

[15] A. Omotoso, H. Shopeju, A. M. Joshua, S. Oni. *Improving BGE-M3 Multilingual Dense Embeddings for Nigerian Low Resource Languages*. Proceedings of the 9th Widening NLP Workshop (WiNLP), pp. 224–229, 2025.

[16] A. Nzeyimana, A. N. Rubungo. *KinyaColBERT: A Lexically Grounded Retrieval Model for Low-Resource Retrieval-Augmented Generation*. arXiv:2507.03241, 2025.

[17] Z. Navasardyan, S. Bughdaryan, B. Minasyan, H. Davtyan. *Less is More: Adapting Text Embeddings for Low-Resource Languages with Small Scale Noisy Synthetic Data*. arXiv:2603.22290, 2026.

[18] A. Alexandrov, V. Raychev, D. I. Dimitrov, C. Zhang, M. Vechev, K. Toutanova. *BgGPT 1.0: Extending English-centric LLMs to other languages*. arXiv:2412.10893, 2024.

[19] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu. *M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. Findings of the Association for Computational Linguistics: ACL 2024, 2024. arXiv:2402.03216.

[20] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, F. Wei. *Multilingual E5 Text Embeddings: A Technical Report*. arXiv:2402.05672, 2024.

[21] N. Reimers, I. Gurevych. *Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation*. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 4512–4525, 2020.

[22] S. Robertson, H. Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4):333–389, 2009.

[23] G. V. Cormack, C. L. A. Clarke, S. Büttcher. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 758–759, 2009.

[24] R. Nogueira, K. Cho. *Passage Re-ranking with BERT*. arXiv:1901.04085, 2019.

[25] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang. *Lost in the Middle: How Language Models Use Long Contexts*. Transactions of the Association for Computational Linguistics, 12:157–173, 2024.

[26] Q. McNemar. *Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages*. Psychometrika, 12(2):153–157, 1947.

[27] M. D. Smucker, J. Allan, B. Carterette. *A Comparison of Statistical Significance Tests for Information Retrieval Evaluation*. Proceedings of the 16th ACM Conference on Information and Knowledge Management (CIKM), pp. 623–632, 2007.

[28] Y. Zhang и др. *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. arXiv:2506.05176, 2025.

[29] H. Schechter Vera и др. *EmbeddingGemma: Powerful and Lightweight Text Representations*. arXiv:2509.20354, 2025.

[30] R. J. Connor. *Sample Size for Testing Differences in Proportions for the Paired-Sample Design*. Biometrics, 43(1):207–211, 1987.

[31] A. Holtzman, J. Buys, L. Du, M. Forbes, Y. Choi. *The Curious Case of Neural Text Degeneration*. International Conference on Learning Representations (ICLR), 2020. arXiv:1904.09751.

[32] E. B. Wilson. *Probable Inference, the Law of Succession, and Statistical Inference*. Journal of the American Statistical Association, 22(158):209–212, 1927.

ПРИЛОЖЕНИЯ

Приложение 1. Инсталиране и стартиране

Командите са за Windows и се изпълняват в PowerShell или в Git Bash от основната папка на проекта. Необходими са Python 3.11 и около 20 GB свободно място за моделите и библиотеките. За работа на графичния процесор е нужна видеокарта NVIDIA с драйвер, поддържащ CUDA 12.6. Без нея системата работи на централния процесор, но повторното класиране е бавно и в интерфейса трябва да се изключи. Google Chrome е нужен само за скрипта за проверка на приложението.

1. Генеративен модел. Инсталира се Ollama и се изтеглят двата варианта на BgGPT:

```
winget install --id Ollama.Ollama -e
ollama pull hf.co/INSAIT-Institute/BgGPT-Gemma-3-4B-IT-GGUF:Q4_K_M
ollama pull hf.co/INSAIT-Institute/BgGPT-Gemma-2-2.6B-IT-v1.0-GGUF:
   Q4_K_M
```

2. Среда на Python. Създава се виртуална среда и библиотеките се инсталират на отделни стъпки, защото общата инсталация изчерпа паметта на машината за разработка при разархивирането на PyTorch. Втората команда заменя варианта на PyTorch само за централния процесор с варианта с CUDA и се пропуска, ако няма видеокарта NVIDIA:

```
python -m venv .venv
.venv/Scripts/python.exe -m pip install sentence-transformers
.venv/Scripts/python.exe -m pip install "torch==2.14.0+cu126" --index
   -url https://download.pytorch.org/whl/cu126
.venv/Scripts/python.exe -m pip install faiss-cpu ollama streamlit
   rank_bm25
.venv/Scripts/python.exe -m pip install -e .
```

3. Индекси и модели. Индексите се изграждат еднократно, като при това се изтеглят моделите за векторно представяне. Последната команда изтегля модела за повторно класиране. Изграждането на индекса с `BAAI/bge-m3` отнема около две минути на графичния процесор и около два часа на централния. Тази стъпка изисква връзка с интернет, а след нея системата работи без мрежа:

```
.venv/Scripts/python.exe -m medguiderag.indexing.build_index "BAAI/
   bge-m3"
.venv/Scripts/python.exe -m medguiderag.indexing.build_index "
   intfloat/multilingual-e5-base"
.venv/Scripts/python.exe -m medguiderag.indexing.build_index "
   sentence-transformers/paraphrase-multilingual-mpnet-base-v2"
.venv/Scripts/python.exe -c "from sentence_transformers import
   CrossEncoder; CrossEncoder('BAAI/bge-reranker-v2-m3', device='cpu
   ')"
```

4. Стартиране. Приложението се стартира с командата по-долу и се отваря на адрес `http://localhost:8501`. Първият въпрос отнема около 35 секунди, докато се заредят моделите. На машина с малко свободна памет е препоръчително броят на нишките на числовите библиотеки да се ограничи с променливите на средата `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` и `MKL_NUM_THREADS` (например до 2):

```
.venv/Scripts/python.exe -m streamlit run src/medguiderag/app/ui.py
```

5. Възпроизвеждане на оценката. Скриптовете за оценка се изпълняват при спряно приложение с `python -m evaluation.<script>`. Описание на всеки от тях и на таблицата, която възпроизвежда, има във файла `evaluation/README.md`.